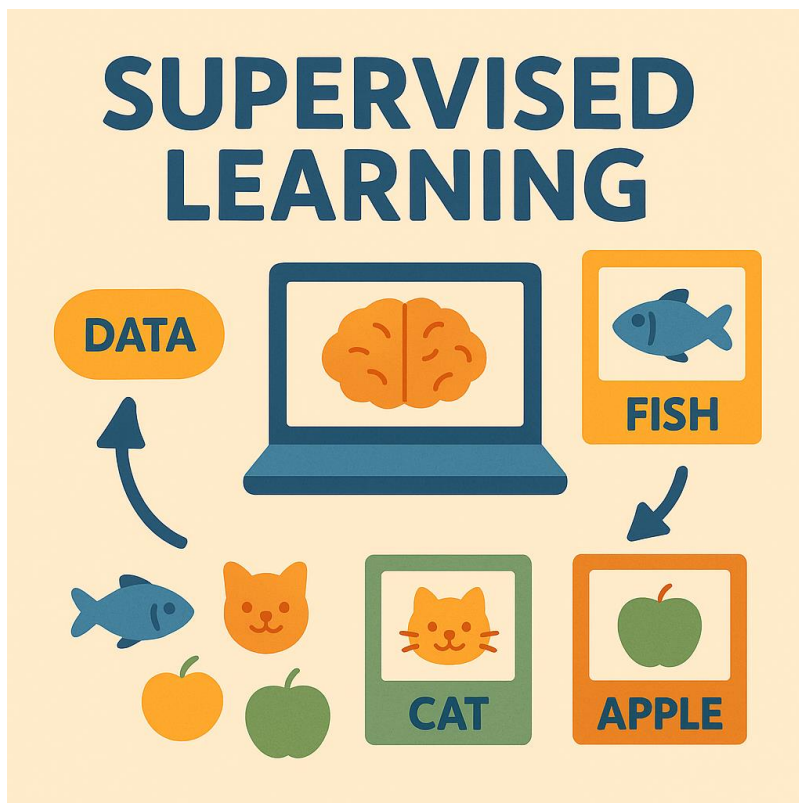


NF2 – MODELS SUPERVISATS



Autor: Generada ChatGPT

Curs d'Especialització en Intel·ligència Artificial i BigData

MP5072 Sistemes d'aprenentatge automàtic

Continguts

Introducció.....	6
Models Supervisats de Regressió.....	7
Regressió lineal simple.....	8
Avaluar el model de regressió lineal simple	11
MAE (Mean Absolute Error) – Error mig absolut.....	12
MSE (Mean Square Error) – Error quadràtic mig (ECM)	12
RMSE (Root Mean Square Error) – Arrel de l’error quadràtic mig	12
R-quadrat (R^2) <i>R Squared</i>	13
Regressió lineal múltiple	19
Exhaustiu (All-in)	20
Eliminació cap enrere (Backward elimination technique)	20
Selecció endavant (<i>Forward Selection</i>)	23
Eliminació bidireccional (Bidirectional elimination / Stepwise)	25
Tots els models possibles.....	26
Avaluar el model de regressió lineal múltiple.....	27
R^2 ajustat (R_a^2).....	27
Regressió polinòmica	33
Avantatges d’ús de la regressió polinòmica.....	37
Problemes de la regressió polinòmica	37
Models lineals regularitzats	39
Regularització L1 (<i>Lasso Regression</i>).....	40
Això fa que alguns coeficients es converteixin exactament en zero , fent que el model elimini variables no rellevants.	40
Regularització L2 (<i>Ridge Regression</i>)	42
Regularització xarxa elàstica (<i>Elastic Net</i>) – L1 +L2.....	43

Resum	43
Configuració hiperparàmetre lambda (λ)	45
Suport Vector Machines	47
Decision Trees (Regressió)	48
Arbres de decisió.....	48
Arbres de regressió	49
Això és exactament el que fa un arbre de regressió.....	49
Algorisme Recursive Binary Splitting	51
Predicció amb un arbre de regressió	52
Models Combinats (<i>Ensemble Models</i>).....	54
<i>Bagging</i> (<i>Bootstrap Aggregating</i>).....	54
<i>Boosting</i> (Potenciació seqüencial)	55
<i>Stacking</i> (Generalització Apliada)	55
Random Forest for Regression.....	56
Extra-Trees	61
Gradient Boosting Regressor	62
Binning	64
Early Stopping	65
XGBoost i LightGBM	66
LightGBM	67
XGBoost Regressor.....	68
Altres models Gradient Boosting	69
Resum Ensemble Models.....	70
Stacking - Exemple	70
Resum General de models de Regressió.....	70
Models Supervisats de Classificació.....	72

Avaluar els models de classificació	72
<i>Accuracy</i> (Exactitud).....	73
<i>Precision</i> (Precisió)	75
<i>Recall</i> (Sensibilitat / Exhaustivitat).....	76
Specificity (especificitat)	77
F1 Score.....	78
Resum	78
Compensació precisió/recall.....	79
Corva ROC i AUC.....	82
Regressió logística.....	85
Funció sigmoide	86
Construcció del model	87
Multivariable.....	87
Recomanacions	87
Naive Bayes	88
Probabilitat.....	88
Distribució conjunta.....	89
Distribució marginal.....	90
Distribució condicional.....	91
Teorema de Bayes.....	93
Naive Bayes	95
Laplace Smoothing.....	97
Tipus de classificadors Naive Bayes	98
Punts forts i febles	98
Decision Trees (Classifier)	99
Classification Error Rate (CER)	100

Índex de Gini (criteri per defecte a CART i scikit-learn)	101
Entropia de Shannon (Cross-Entropy / Information Gain).....	106
Quan utilitzar Gini o Entropy?.....	110
Random Forest Classifier	111
KNN (K-Nearest Neighbors)	113
Tipus de distàncies	114
Quin valor de K escollim?	117
ANNEX	121
REFERÈNCIES	121

Introducció

L'aprenentatge supervisat és un tipus d'aprenentatge automàtic (Machine Learning) en què el model aprèn a partir d'exemples ja etiquetats.

És com un estudiant que aprèn fent exercicis amb les respostes al costat: veu les preguntes (les característiques o *features*) i sap quina és la resposta correcta (l'etiqueta o *target*).

Objectiu: crear un model capaç de predir correctament les etiquetes d'exemples nous, que no ha vist abans.

Formalment l'objectiu és aprendre una funció:

$$f(X) = y$$

On:

- **X** = conjunt de característiques d'entrada (**variables independents**)
- **y** = variable objectiu o etiqueta (**variable dependent**)

El tipus d'aprenentatge podrà ser de regressió o de classificació depenent de la variable independent / objectiu.

Tipus	Tipus de sortida	Exemples
Regressió	Valor numèric continu	Predir el preu d'una casa, la temperatura, la facturació
Classificació	Categoria	Diagnòstic (mallat/sà), predicció de frau (sí/no). Predir la categoria de d'una flor Iris

Models Supervisats de Regressió

La **regressió** és una de les tasques més fonamentals dins de l'aprenentatge supervisat. El seu objectiu principal és predir un valor numèric continu a partir d'una o més variables d'entrada. A diferència de la classificació, on el model assigna etiquetes discretes com "Sí/No" o "Aprovats/Suspesos", **la regressió busca estimar quantitats**: quant costarà un producte, quina serà la temperatura demà o quin consum energètic tindrà un edifici segons les seves característiques.

Podem entendre la regressió com un procés mitjançant el qual un algorisme aprèn la relació que hi ha entre diverses variables per poder-ne predir una altra. El model intenta construir una funció que representi el vincle entre les dades d'entrada i la sortida esperada. Per exemple, si coneixem la mida, el nombre d'habitacions i la ubicació d'un pis, el model pot mirar d'estimar-ne el preu de mercat. Aquesta capacitat de **capturar relacions quantitatives**.

Començarem aprenent la **Regressió Lineal**, el model més senzill i alhora més fonamental dins d'aquest conjunt de tècniques. Comprendre la regressió lineal és essencial, perquè serveix de base per entendre conceptes més avançats com la relació entre variables, la mesura de l'error o la idea d'ajustar un model a un conjunt de dades. A més, és un model fàcilment interpretable: permet entendre de quina manera cada variable d'entrada influeix en el resultat final.

Ara bé, La regressió lineal no és l'única eina disponible. Molts altres algorismes, inicialment pensats per a classificació, poden adaptar-se per resoldre problemes de regressió amb petites modificacions. És el cas dels **àrbre de decisió (Decision Trees)**, dels **models d'ensamblatge** com **Random Forest** o **Gradient Boosting**, o de les **màquines de vectors suport (Support Vector Machines)**, que en la seva versió **SVR (Support Vector Regression)** permeten treballar amb valors continus.

També existeixen altres mètodes pensats específicament per a regressió, com els **models polinòmics**, que permeten capturar relacions corbades o no lineals; les regressions amb **regularització**, com **Ridge** o **Lasso**, que ajuden a evitar el sobreajustament.

Hi ha tècniques especialitzades per a **sèries temporals**, com **ARIMA** o **Prophet**, especialment útils quan les dades depenen del temps. Encara que aquestes tècniques no s'estudien en profunditat en aquest

mòdul, conèixer-ne l'existència ajuda a triar millor el model més adequat segons el tipus de problema i les característiques de les dades.

El repte de construir un bon model de regressió no consisteix únicament a obtenir prediccions precises. Un model realment útil ha de ser capaç de **generalitzar bé**, és a dir, de funcionar correctament amb dades noves que no ha vist durant l'entrenament. També ha de ser **interpretable**, perquè en entorns professionals sovint és tan important entendre el "per què" el model fa una predicció, com la predicció mateixa. A més, cal que s'adapti al **context d'ús**: no és el mateix predir el preu d'un cotxe de segona mà que el risc financer d'una inversió, i cada cas requereix un enfocament específic.

Podem imaginar la regressió com el procés d'un meteoròleg que vol predir la temperatura de demà. Té dades històriques de dies anteriors —com la humitat, la pressió atmosfèrica o el vent— i intenta descobrir una relació entre aquestes variables i la temperatura final. Si aconsegueix capturar correctament aquest patró, podrà predir amb força precisió el que passarà demà. Aquesta és, en essència, la idea darrere de qualsevol model de regressió: aprendre **com les variables explicatives determinen una resposta numèrica**.

En resum, la regressió és una de les pedres angulars del *Machine Learning* perquè permet predir valors continus, analitzar relacions entre variables i comprendre millor el comportament de sistemes complexos. Dominar-ne el funcionament és un pas imprescindible per desenvolupar models predictius fiables, capaços de generalitzar i de donar suport a la presa de decisions en entorns reals.

Regressió lineal simple

La **regressió lineal simple** és, sens dubte, el model de regressió més senzill i alhora més fonamental dins del *Machine Learning*. La seva missió és representar la relació que hi ha entre una variable d'entrada (o predictor) i una variable de sortida (o objectiu) mitjançant una **línia recta**.

Podem pensar-hi com intentar ajustar una recta que "passi pel mig" dels punts d'un núvol de dades. Aquesta recta descriu la tendència general: quan la variable X augmenta, com es comporta Y? Creix també? Disminueix? Es manté estable?

El model matemàtic és molt senzill i respon a una equació de primer grau:

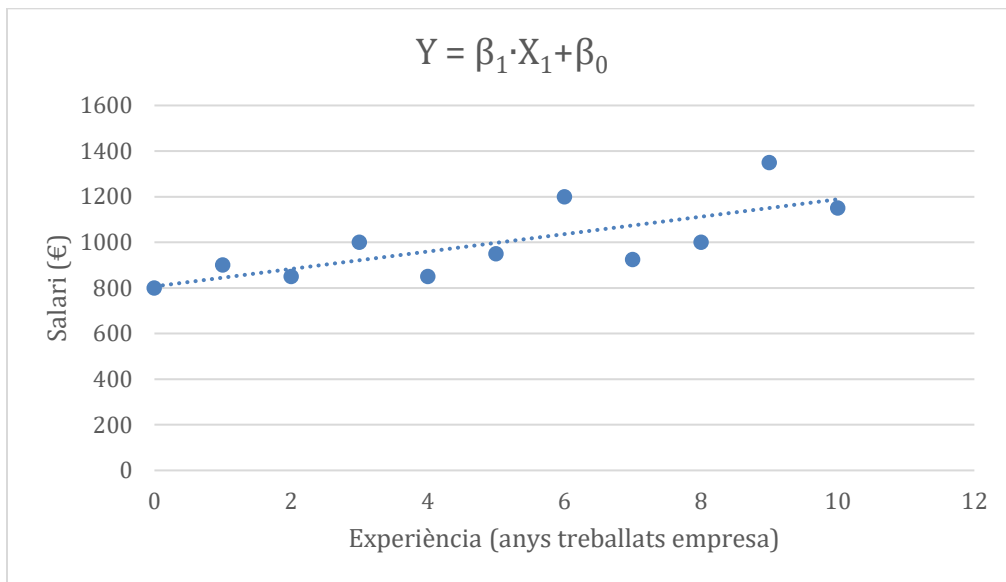
$$Y = \beta_1 \cdot X_1 + \beta_0$$

Y = variable dependent (VD). Per exemple el sou dels empleats.

X_1 = variable independent (VI). Aquesta variable farà variar la variable dependent. Per exemple els anys que el treballador porti treballant a l'empresa.

β_1 = És la pendent de la recta o el coeficient. Indica quan s'ha d'incrementar el sou per cada any que el treballador està a l'empresa.

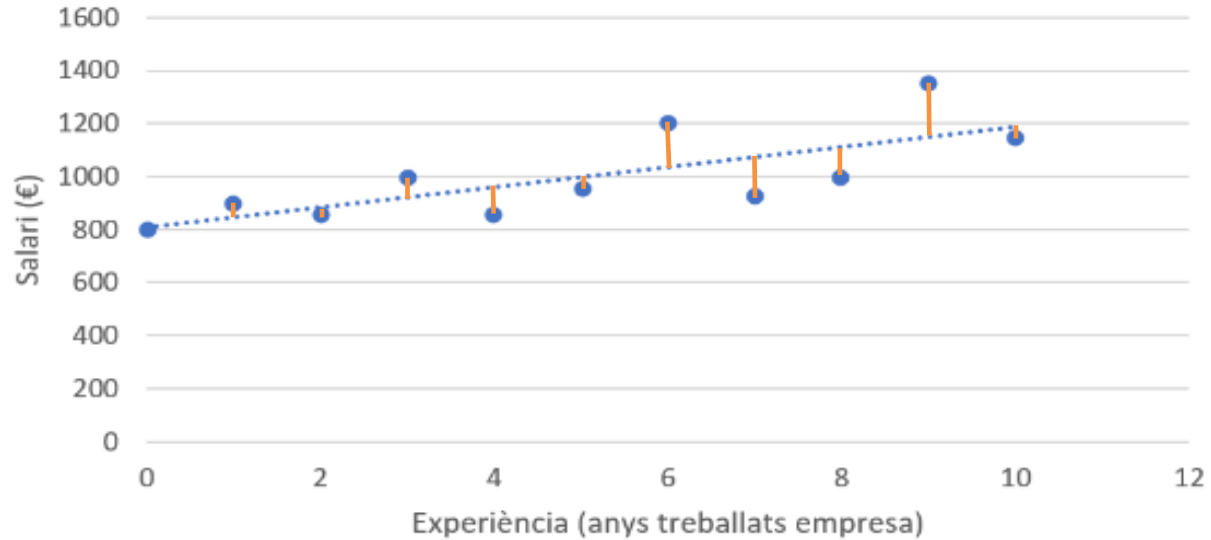
β_0 = Constant. El valor inicial que agafarà la variable dependent (VD) si no existís la variable independent (VI).



$$\text{Salari} = Y = \beta_0 + \beta_1 \cdot \text{Experiència}$$

Si ens hi fixem en la β_1 (pendent/coeficient) veiem que ens indica per cada any quin és l'increment de salari.

Per trobar la recta podem utilitzar el **mètode del mínims quadrats (Least Squares Regression)**



Els punts blaus representen les observacions reals que tenim de les dades i les línies verticals de color verd que van de les observacions reals fins a la línia de regressió són la diferència del que diu el nostre model i la dada real.



En aquest punt veiem que hi ha una diferència de la dada real Y_i i la dada estimada del nostre model $\hat{Y}_i$. El que hem de mirar és que la longitud de la línia verda entre Y_i (valor real) i $\hat{Y}_i$ (valor estimat) sigui el més petit possible. Això és el que fa precisament el mètode dels mínims quadrats.

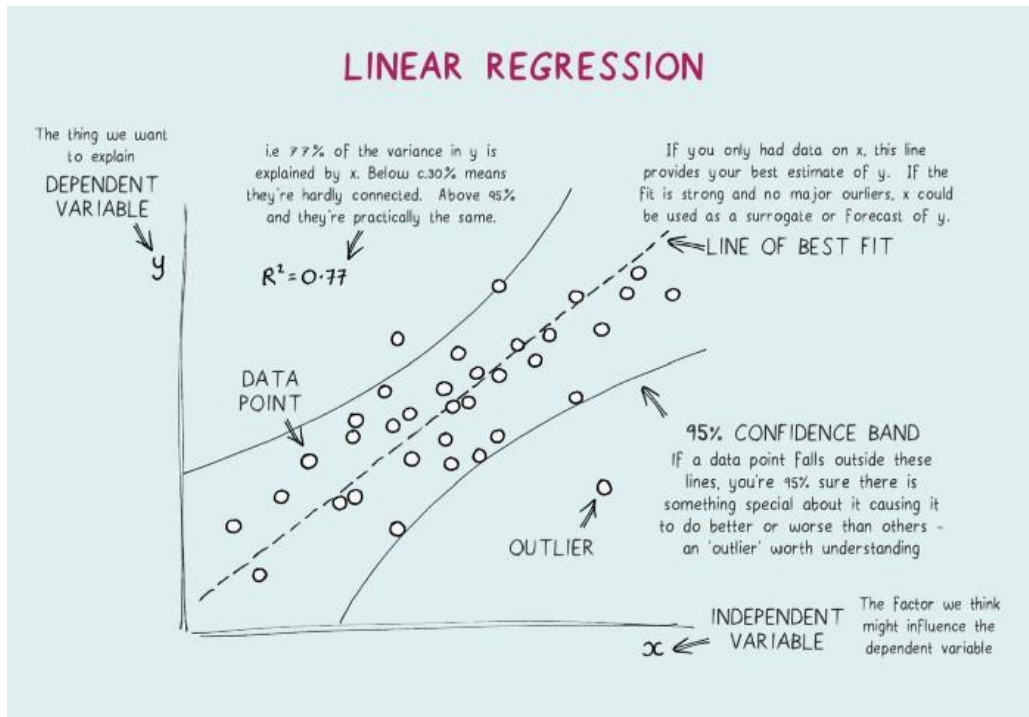
$$\min \sum_i (y_i - \hat{y}_i)^2$$

Sumar totes les diferències. Les elevem al quadrat perquè tots siguin positives i llavors les sumem. Aquestes sumes de diferències el quadrat. De totes les rectes ens quedarem en aquella que aquest valor sigui el més petit.

Minimitza els quadrats de les diferències entre la dada real i la predicció.

El model trobarà el coeficient β_1 i la constant la β_0

Altres mètodes podrien ser **el mètode de màxima versemblança (Maximum Likelihood Estimation) o mitjançant el descendent del gradient.**



Il·lustració 1 By Dan White 1000 from Shutterstock

Avaluar el model de regressió lineal simple

Encara que hem ajustat un model és necessari verificar la seva eficàcia, ja que encara que la línia que millor s'ajusti a les observacions entre totes les possibles, el model pot ser dolent.

Les mètriques les utilitzarem per avaluar els models i veure si aquests són bons o generalitzaren bé.

Existeixen multitud de mètriques, però depenent del tipus de problema utilitzarem unes o unes altres.

	Edats		
	Valor real (y)	Valor predit ($\hat{y}$)	Diferència valor absolut
	32	35	3
	23	28	5
	85	95	10
Mitjana	46,66667	52,66667	6

Considerem la diferència entre un valor predit i un de real com l'error, però com que els valors "reals" es basen en observacions (que poden tenir variància aleatòria) direm que la diferència és entre valor

predit ($\hat{y}$) i un valor observat (y). En aquest cas en termes matemàtics i/o estadístics la diferència s'anomena **residual**.
Podem resumir els residuals de totes les prediccions per calcular la **pèrdua** general del model com a mesura del seu rendiment predictiu

MAE (Mean Absolute Error) – Error mig absolut

També conegut com a funció de pèrdua L1.

$$\frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$
$$MAE = \frac{1}{3} (|32 - 35| + |23 - 28| + |85 - 95|) = \frac{1}{3} (3 + 5 + 10) = 6$$

MSE (Mean Square Error) – Error quadràtic mig (ECM)

També conegut com a funció de pèrdua L2.

Penalitzta els errors grans ja que elevem el quadrat abans de fer la mitjana.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$
$$MSE = \frac{1}{3} ((32 - 35)^2 + (23 - 28)^2 + (85 - 95)^2) = \frac{1}{3} (3^2 + 5^2 + 10^2) = 44.666$$

És bó el resultat de 44.66?

És difícil de saber perquè el valor no s'expressa en una unitat de mesura significativa. Sabem que quan menor sigui el valor, millor serà el nostre model. Aquest valor és útil per comparar dos models i trobar quin té millor rendiment, però no el podem sospesar massa perquè el valor no s'expressa en una unitat de mesura significativa.

RMSE (Root Mean Square Error) – Arrel de l'error quadràtic mig

Igual que l'anterior, però la diferència entre aquest (RMSE) i l'anterior (MSE) és que fem l'arrel quadrada al final.

Aquesta mètrica és útil per expressar la pèrdua amb les mateixes unitats que el propi valor de la predicció. En aquest cas les edats

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$
$$RMSE = \sqrt{\frac{1}{3} (3^2 + 5^2 + 10^2)} = 6.683$$

Si valorem el valor RMSE (6.683) i el de MAE (6) veiem que el primer empitjorem perquè la última diferència (10) l'elevem al quadrat per donar-li més importància.

Compte amb els valors de MAE i RMSE perquè moltes vegades quan prediem resultats esbiaixats l'error també tindrà aquest biaix. Si tenim un error amb molt biaix la mitjana ens pot dir poca cosa.

R-quadrat (R^2) *R Squared*.

R-quadrat és una de les mesures més utilitzades per mesurar la qualitat de l'ajust. El coeficient R^2 és la **correlació** entre els valors reals i les prediccions. Això tindrà un valor entre 0 i 1. El que volem és aconseguir tenir un valor més pròxim a 1 possible.

Quan més gran sigui la R-quadrat (R^2) més bó serà el nostre model ja que tots els punts cauen a dins de la nostra recta de regressió.

En una regressió lineal el que fem és buscar el mínim de les distàncies entre els valors predits i els reals:

$$\text{minimitzar} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Suma dels quadrats dels residus

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Suma dels quadrats totals

Aquest factor no depèn del model sinó que simplement avalua les diferències entre les coordenades y i la mitjana de cadascun d'ells

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Veien a la fórmula que R^2 tendirà a 1 quan SS_{res} (la suma dels quadrats dels residus) sigui més petit o vagi cap al 0 (zero).

Una altra interpretació seria dir com de bo és el nostre model si agaféssim SS_{tot} (suma dels quadrats totals) com a recta de regressió.

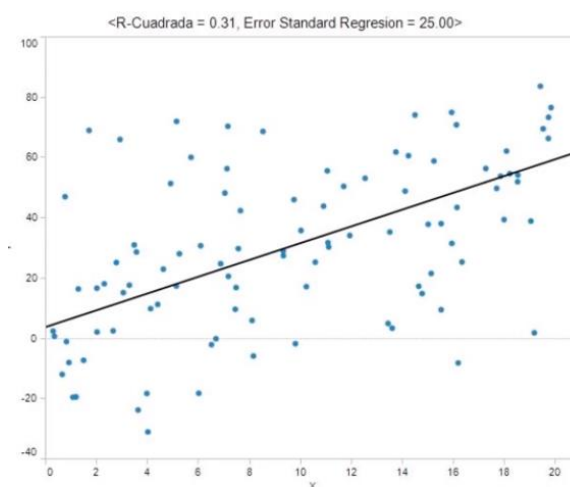
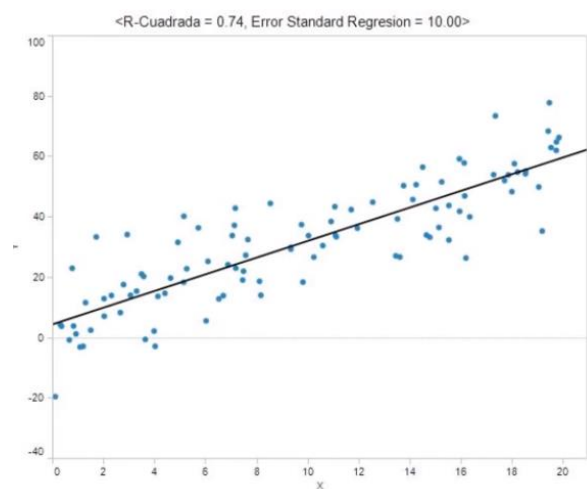
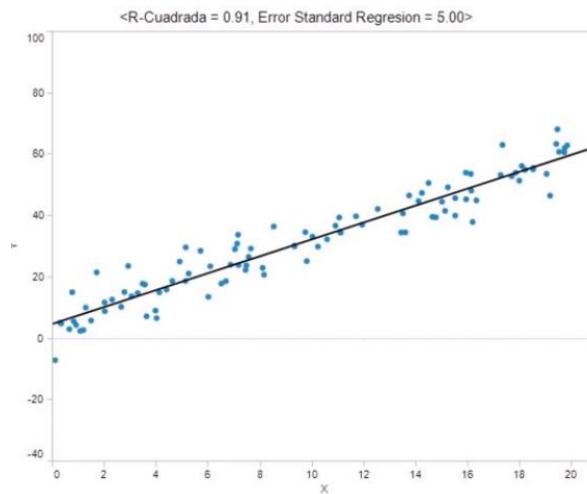
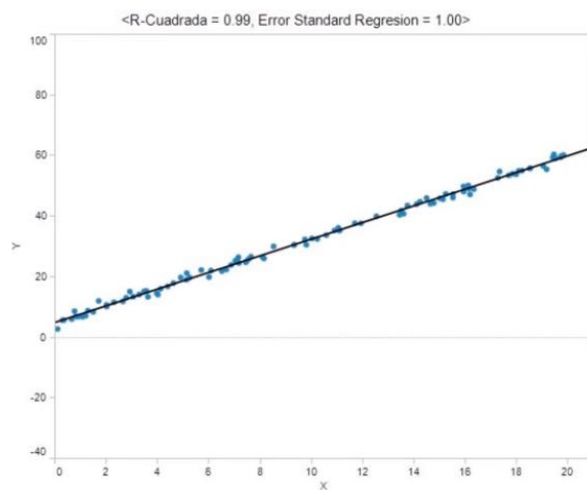
R^2 podria ser negatiu? Si, però això significaria que el model trobat és pitjor que si agafem la mitjana com a valor de predicció.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

En l'exemple:

$$R^2 = 1 - \frac{((32 - 35)^2 + (23 - 8)^2 + (85 - 95)^2)}{((32 - 46.67)^2 + (23 - 46.67)^2 + (85 - 46.67)^2)} = 1 - \frac{134}{2.244,667} = 0,940$$

Fixem-nos en els següents gràfics.



Overfitting en la Regressió

Per veure si ens trobem davant d'un cas o no d'overfitting en un problema de regressió és mirant R^2 .

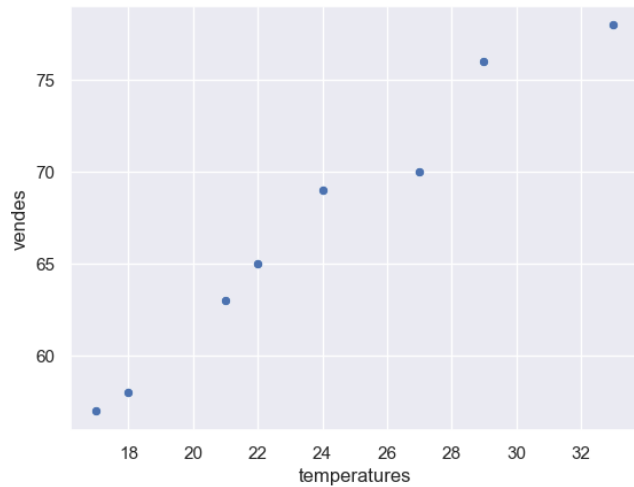
Cas	R^2 train	R^2 test	Comentari
Overfitting	0.98	0.65	Molta diferència entre <i>train/test</i>
Bona generalització	0.70	0.68	Poca diferència i <i>train</i> millor.
Underfitting	0.50	0.45	Poca diferència, però valor R^2 baix.
Regularització o aleatorietat	0.75	0.80	Poca diferència, però <i>test</i> millor

Exemple – Venda de gelats

Un establiment vol predir la venda de gelats que tindrà en un dia determinat per poder aprovisionar-se adequadament. [Enllaç GitHub](#)



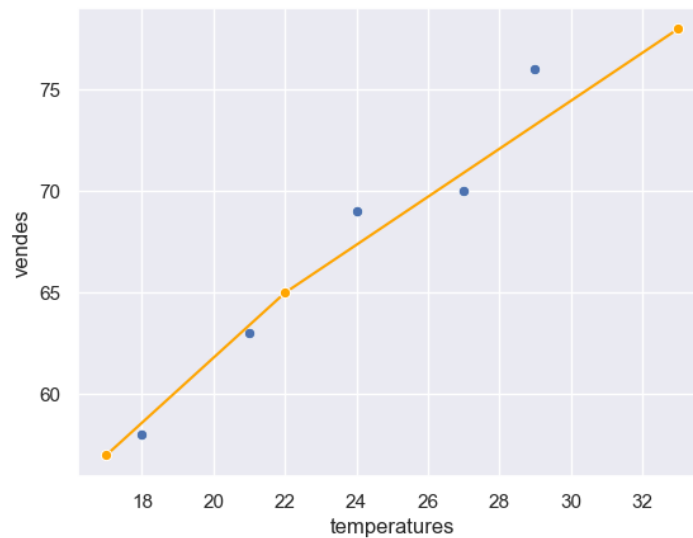
	Temperatura	Vendes
ENTRENAMENT	18	58
	21	63
	24	69
	27	70
	29	76
VALIDACIÓ	33	78
	17	57
	22	65



Veiem que existeix una suposada relació lineal entre x (temperatura) i y (vendes).

Si el model obtingut mitjançant l'entrenament és el següent $Y = 1.52538X + 30.89594$

Podem veure la comparació dels valor de y i $\hat{y}$

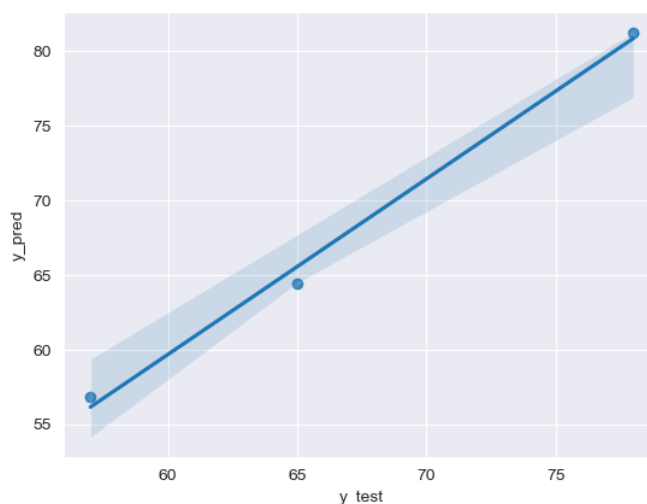


Temperatura x	Vendes y	Vendes $\hat{y}$	$ y - \hat{y} $	$(y - \hat{y})^2$	$(y - \bar{y})^2$
18	58				
21	63				
24	69				
27	70				
29	76				
18	58	81.2334	3.2334	10.4553	128
21	63	56.8274	0.1726	0.0297	
24	69	64.4543	0.5457	0.2977	
SUMA	200		3.95178	10.7829	
MITJANA	66.6667		1.31726	3.5944	

$$MSE = \frac{1}{n} \sum_{i=1}^n (\gamma_i - \hat{\gamma}_i)^2 = 3.5944$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\gamma_i - \hat{\gamma}_i)^2} = \sqrt{MSE} = \sqrt{3.5944} = 1.8971$$

$$R^2 = 1 - \frac{\sum (\gamma_i - \hat{\gamma}_i)^2}{\sum (\gamma_i - \bar{\gamma})^2} = 1 - \frac{10.7829}{224.66667} = 0,95200$$



Veiem amb l'avaluació del model que tenim un coeficient de determinació R^2 força bó amb un valor de **0,952**.

Scatter plot de y_{pred} i y_{test} sembla que també compleix una recta a on no hi ha massa error.

Exercici Beisbol

Volem analitzar si existeix una relació entre el número de batejos (que realitza un equip de beisbol i el número d'entrades que aconsegueix.

En cas d'existir aquesta relació caldria establir un model per poder predir el resultat d'un partit.

Regressió lineal múltiple

En una regressió lineal múltiple també tenim una variable depenent (VD) i aquesta la prediem a través de moltes altres variables independents (VI).

En la regressió lineal múltiple no ens conformem amb una sola variable explicativa, sinó que volem entendre com diversos factors junts influeixen sobre una mateixa variable objectiu. És el cas real més habitual: les vendes d'una empresa depenen del preu, de la publicitat, de la temporada, de la competència...

Per exemple, la nota d'un estudiant no només ve determinada per les hores d'estudi, sinó que pot venir influenciada per altres variables: hores de descans, hores que ha assistit a classe,... Cadascuna d'aquestes variables tindrà un coeficient ($a_1, a_2, \dots, a_n$).

$$Y = \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \beta_0$$

El primer que hem de tenir en compte és que no totes les variables independents seran útils o ens aportaran informació al model.

$$Y = \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \beta_0$$

No és bó quedar-nos amb totes les variables perquè a vegades no per tenir més variables tenim més informació o el model serà millor, sinó que a vegades tenim un model que prediu el mateix en poques variables que amb moltes.

Un altre problema afegit és que quan haguem de defensar el model si hi ha moltes variables costarà molt defensar-lo i explicar com funciona.

Com podem fer per seleccionar les variables correctes? Doncs hi ha diferents mètodes:

Exhaustiu (*All-in*)

Eliminació cap enrere (*Backward elimination technique*)

Selecció endavant (*Forward Selection*)

Eliminació bidireccional (*Bidirectional elimination*)

Comparació de puntuacions (*scores*)

Quina de les tècniques anteriors és la millor? Doncs depèn. No totes donaran el mateix model. Per tant recordem que el nostre objectiu alhora de construir un model de regressió és reduir el màxim l'error de quadrats (SSE) sense caure en **overfitting**.

Hem de poder crear un model en menys variables possibles per definir la variable depenent.

Si afegim més variables independents creem més relacions entre ells. Per tant podem caure a la **multicol·linealitat**. Per tant, el cas ideal és que totes les variables independents estiguin correlacionades amb la variable depenent, però no entre elles.

Exhaustiu (All-in)

És quan posem totes les variables en el model, però ja hem dit que no és una bona pràctica.

L'utilitzarem si tenim coneixement previ i ja sabem que totes les variables han d'estar a dins del model.

Això és perquè tenim algun expert que sap que s'han d'utilitzar totes.

Un altra opció per introduir totes les variables al model és quan ens ho exigeixen.

Preparació prèvia per realitzar el mètode de l'eliminació cap enrere (*backward elimination*).

Eliminació cap enrere (Backward elimination technique)

La tècnica d'eliminació cap enrere és quan agafem totes les variables independents per generar el model i anem eliminant-ne un a una mentre hi hagi alguna variable amb un p-valor > a un cert un nivell de significació α .



Passos:

- **PAS 1:** Establim el nivell de significació (SL/α) per estar dins el model ($\alpha \leq 0.05$)
- **PAS 2:** Calculem el model amb totes les possibles variables independents
- **PAS 3:** Agafem la variable independent amb el p-valor més gran
 - Si p-valor > α llavors passem al PAS 4
 - Altrament passem al PAS 5 - Fi
- **PAS 4:** Eliminem la variable predictora que té el p-valor més gran i tornem a calcular el model
- **PAS 5:** Fi

Exemple:

Tenim les variables X_1, X_2, X_3, X_4, X_5 ,

PAS 1: Establim el nivell de significació a 0.05

PAS 2: Calculem el model amb totes les possibles variables independents.

$$Y = \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4 + \beta_5 X_5 + \beta_0$$

PAS 3: Comprovem el p-valor de totes les variables independents i agafem la més alta i la comparem amb α

- **p-valor_{x1} = 0.0**
- **p-valor_{x2} = 0.691**
- **p-valor_{x3} = 0.155**
- **p-valor_{x4} = 0.943**
- **p-valor_{x5} = 0.0**

El p-valor de X_4 (0.943) és superior al de α (0.05). Per tant traiem anem al PAS 4.

PAS 4: Eliminem la variable X_4 del model i anem al PAS 2

PAS 2: Calculem el model

$$Y = \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_5 X_5 + \beta_0$$

PAS 3: Comprovem el p-valor de totes les variables independents i agafem la més alta i la comparem amb α

- **p-valor_{x1} = 0.0**
- **p-valor_{x2} = 0.674**
- **p-valor_{x3} = 0.155**
- **p-valor_{x5} = 0.0**

El p-valor de X_2 (0.674) és superior al de α (0.05). Per tant traiem anem al PAS 4.

PAS 4: Eliminem la variable X_2 del model i anem al PAS 2

PAS 2: Calculem el model

$$Y = \beta_1 X_1 + \beta_3 X_3 + \beta_5 X_5 + \beta_0$$

PAS 3: Comprovem el p-valor de totes les variables independents i agafem la més alta i la comparem amb α

- **p-valor_{x1} = 0.0**
- **p-valor_{x3} = 0.04**
- **p-valor_{x5} = 0.0**

El p-valor de X3 (0.04) és inferior al de α (0.05). Per tant, ja estem i anem al PAS 5

PAS 5: Ens quedem amb el següents model

$$Y = \beta_1 X_1 + \beta_3 X_3 + \beta_5 X_5 + \beta_0$$

Vídeos:

- [Statistics 101: Multiple Regression, Backward Elimination](#), *Brandon Foltz*

Selecció endavant (*Forward Selection*)

Aquest mètode no és l'efecte contrari o els passos del mètode anterior invertits.



Passos:

- **PAS 1:** Establir el nivell de significació (SL/α) per entrar a dins el model ($\alpha = 0.05$)
- **PAS 2:** Calcular tots els models de regressió amb cadascuna de les variables independents i ens quedarem amb el model que tingui el **p-valor més petit**.
- **PAS 3:** Conservarem la variable independent del pas anterior i ajustem tots els possibles models incloent aquesta variable independent
- **PAS 4:** Considerem la variable independent amb el menor p-valor.
Si $p\text{-valor} < \alpha$ llavors passem al PAS 3ç
Altrament passem al PAS 5 - Fi.
- **PAS 5:** Fi

Exemple:

Tenim les variables X_1, X_2, X_3, X_4, X_5 ,

PAS 1: Establim el nivell de significació a 0.05

PAS 2: Calculem tots els models amb una sola variable independent.

$$Y_1 = \beta_1 X_1 + \beta_0$$

$$Y_2 = \beta_2 X_2 + \beta_0$$

$$Y_3 = \beta_3 X_3 + \beta_0$$

$$Y_4 = \beta_4 X_4 + \beta_0$$

$$Y_5 = \beta_5 X_5 + \beta_0$$

El model que té el menor p-valor és Y_2 amb un valor de 0.0097

PAS 3: Conservem X_2 i ajustem tots els models incloent les altres variables independents.

$$Y_{21} = \beta_1 X_1 + \beta_2 X_2 + \beta_0$$

$$Y_{23} = \beta_3 X_3 + \beta_2 X_2 + \beta_0$$

$$Y_{24} = \beta_4 X_4 + \beta_2 X_2 + \beta_0$$

$$Y_{25} = \beta_5 X_5 + \beta_2 X_2 + \beta_0$$

El model que té el menor p-valor és Y_{25} amb un valor de 0.035

PAS 4: Ens quedem amb el model de Y_{25} i recalculem altre cop tots els models amb totes les variables

$$Y_{251} = \beta_1 X_1 + \beta_2 X_2 + \beta_5 X_5 + \beta_0$$

$$Y_{253} = \beta_3 X_3 + \beta_2 X_2 + \beta_5 X_5 + \beta_0$$

$$Y_{254} = \beta_4 X_4 + \beta_2 X_2 + \beta_5 X_5 + \beta_0$$

El model que té el menor p-valor és Y_{254} amb un valor de 0.073

Com que el p-valor $> \alpha$ Finalitzem i acabem el procés agafant el model Y_{25}

Una altra variant d'aquesta tècnica és utilitzar el valor de R^2 en el PAS2 per quedar-nos amb el model que millor s'ajusti. Amb un llindar de R^2 especificat, per exemple 0.95.

<https://www.youtube.com/watch?v=ckojLIMPtLw>

Vídeos:

[Statistics 101: Multiple Regression, Forward Selection](#), *Brandon Foltz*

Eliminació bidireccional (Bidirectional elimination / Stepwise)



Aquest mètode intenta utilitzar la bondat dels dos anteriors.

- **PAS 1:** Establirem dos nivells de significació. Aquell per entrar al model i aquell per sortir-ne (SLIN/ α_{in} i SLSTAY/ α_{stay}) ($\alpha_{in} = \alpha_{stay} = 0.05$)
- **PAS 2:** Utilitzarem el *Forward Selection* amb les variables noves amb p-valor $< SLIN/\alpha_{in}$
- **PAS 3:** Utilitzarem TOTS els passos del *Backward elimination technique* per mantenir les variables antigues p-valor $< SLSTAY/\alpha_{stay}$
- Repetirem el pas 2 i pas 3 fins que no hi hagi variable per entrar o per sortir.
 - Afegim la millor variable i eliminem les que sobren.
 - Cada vegada que afegim una variable ens assegurem que les variables que es queden són les millors.
- **PAS 4:** No hi ha variables per entrar o per sortir.

Vídeos:

- [Statistics 101: Multiple Regression, Stepwise Selection](#), Brandon Foltz

Tots els models possibles

Aquest mètode també anomenat per força bruta el que fa és calcular tots els models possibles i escollir el que millor s'ajusti.

- **PAS 1:** Seleccionem un criteri alhora d'escollir un model
- **PAS 2:** Generar tots els models possibles de regressió. Si tenim N variables tindrem 2^{N-1} combinacions/models possibles.
- **PAS 3:** Seleccionarem el model amb el millor criteri escollit.

Per exemple amb 10 columnes hauríem de generar 1.023 models!!!!

Avaluar el model de regressió lineal múltiple

R^2 ajustat (R_a^2)

Ja hem vist R^2 . En el cas de regressió lineal múltiple tenim un problema i és que R^2 sempre quedarà igual. O sigui no empitjorà cada vegada que afegim variables.

Com que la suma dels quadrats reals no canvia al fet d'afegir variables, R^2 sempre s'incrementarà o com a mínim no empitjorà. Per tant, si només mirem R^2 podem caure en l'error d'afegir variables al model i “sobre inflar-lo” inútilment

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = R^2 = 1 - \frac{\sum(\gamma_i - \hat{\gamma}_i)^2}{\sum(\gamma_i - \bar{\gamma})^2}$$

Per això en el cas de regressions lineals múltiples ens ajudem del **R^2 ajustat**.

Aquest indicador no només es basa en el nombre de variables del model sinó que busca l'equilibri entre les variables independents i el nombre d'observacions.

L' R^2 ajustat pot arribar a ser negatiu si tenim **moltes variables independents i poques observacions**.

En termes estadístics afegir un variable estem perdent un grau de llibertat, i aquesta pèrdua comporta un cost.

Els graus de llibertat són: $DFE = n - p - 1$

A mesura que anem afegint variables

R^2 ajustat té diferents fórmules per calcular-lo on:

- n : És el número d'observacions
- p : És el número de variables independents

$$R_a^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

$$R_a^2 = 1 - \left(\frac{n - 1}{SS_{tot}} \right) \cdot MSE$$

$$R_a^2 = R^2 - \frac{p(1 - R^2)}{n - p - 1}$$

Exemple 1 (augmentem el número de variables sense augmentar observacions): En el següent exemple veiem que augmentant el nombre de variables de 3 a 100 empitjorem R^2 ajustat



HYPOTHETICAL

$$R_a^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

$p = 3, n = 100, R^2 = 0.7326$	$p = 19, n = 100, R^2 = 0.7651$
$R_a^2 = 1 - (0.2674) \frac{99}{96}$	$R_a^2 = 1 - (0.2349) \frac{99}{80}$
$R_a^2 = 1 - 0.2758$	$R_a^2 = 1 - 0.2907$
$R_a^2 = 0.7242$	$R_a^2 = 0.7093$

Exemple 2 (augmentem el número de variables sense augmentar observacions) : En aquest segon exemple veiem que l'empitjorament ha estat major que l'anterior, perquè el número d'observacions és més petit.



HYPOTHETICAL

$$R_a^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

$p = 3, n = 50, R^2 = 0.7326$	$p = 19, n = 50, R^2 = 0.7651$
$R_a^2 = 1 - (0.2674) \frac{49}{46}$	$R_a^2 = 1 - (0.2349) \frac{49}{30}$
$R_a^2 = 1 - 0.5755$	$R_a^2 = 1 - (0.3837)$
$R_a^2 = 0.7152$	$R_a^2 = 0.6163$

Resum :



HYPOTHETICAL

$$p = 3, n = 100, R^2 = 0.7326 \quad R_a^2 = 0.7242$$

$$p = 19, n = 100, R^2 = 0.7651 \quad R_a^2 = 0.7093$$

$$p = 3, n = 50, R^2 = 0.7326 \quad R_a^2 = 0.7152$$

$$p = 19, n = 50, R^2 = 0.7651 \quad R_a^2 = 0.6163$$

Això ens demostra que sempre hem de mantenir el model tan simple com sigui possible

Mira també els exemples de *Backward* i *Fordward* com evoluciona R^2 i R^2 ajustada

Explicació de la sortida del *summary* de **StatsModels** de Python

- **No. Observacions:** N° d'observacions
- **Df Residuals:** Els grau de llibertat: observacions - variables independents – 1
- **Df Model:** El número de variables independents (excloent el terme independent)

```
lr_ols.summary()
```

OLS Regression Results

Dep. Variable:	Profit	R-squared:	0.952
Model:	OLS	Adj. R-squared:	0.943
Method:	Least Squares	F-statistic:	113.9
Date:	Sun, 04 Dec 2022	Prob (F-statistic):	3.78e-18
Time:	22:23:14	Log-Likelihood:	-370.11
No. Observations:	35	AIC:	752.2
Df Residuals:	29	BIC:	761.5
Df Model:	5		
Covariance Type:	nonrobust		

Càlcul de R^2

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

```
# Calculem els residus dels valors predits amb els reals
residus = y - y_pred

# Calculem la suma dels quadrats dels residus
SSR = np.dot(residus, residus)
print("Suma dels quadrats dels residus (SSR):", SSR)

# Calculem la suma dels quadrats totals
centre_y = y - y.mean()
SST = np.dot(centre_y, centre_y)
print("Suma dels quadrats totals (SST):", SST)

# Calculem R2
R_quadrat = 1 - (SSR / SST)
print("R-quadrat:", R_quadrat)
```

Càlcul de R^2 ajustat

$$R_a^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

```
#Graus de llibertat
df_resid = model.df_resid

# Número d'observacions
nobs = df_med.shape[0]

# Calculem R2 ajustat
R_quadrat_ajust = 1 - (nobs-1/df_resid * (1-R_quadrat))
print("R-quadrat ajustat:", R_quadrat_ajust)
```

Alerta

Regressió polinòmica

Fins ara hem vista dos tipus de regressions lineals:

- Regressió lineal simple: Conté una sola variable independent

$$Y = \beta_1 X_1 + \beta_0$$

- Regressió lineal múltiple: Conté més d'una variable independent

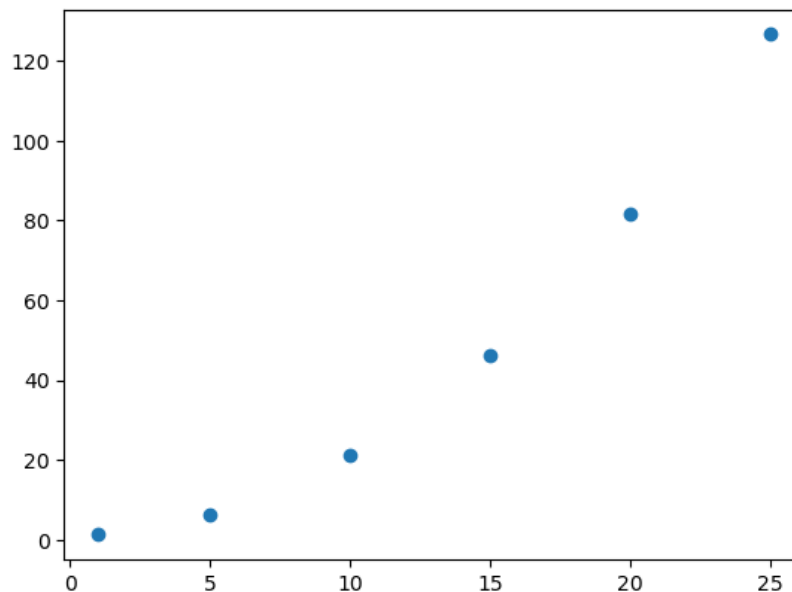
$$Y = \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \beta_0$$

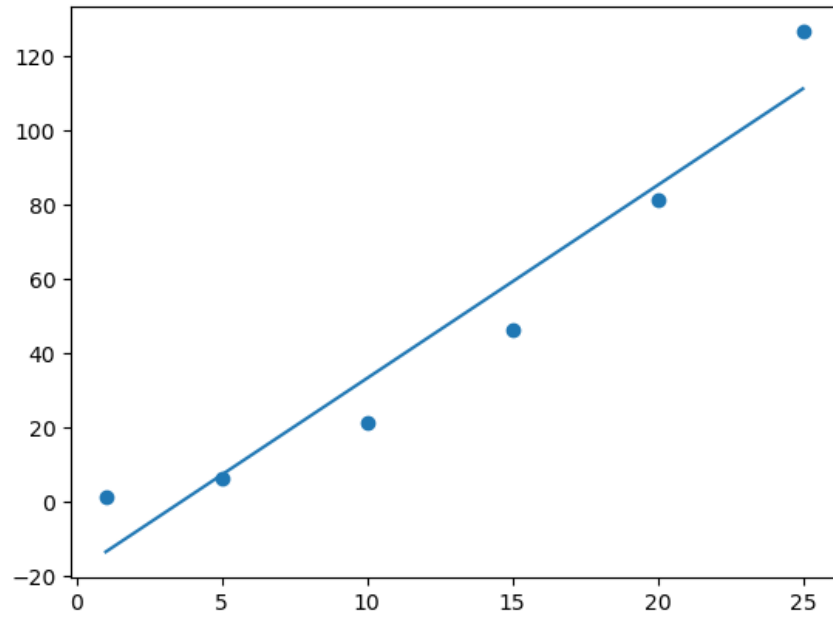
En la regressió polinòmica tenim una sola variable independent, però aquesta expressada en un polinomi de grau n , on aquesta n sigui l'exponent (potència) més elevat de la variable independent.

$$Y = \beta_n X_1^n + \dots + \beta_2 X_1^2 + \beta_1 X_1 + \beta_0$$

Exemple 1

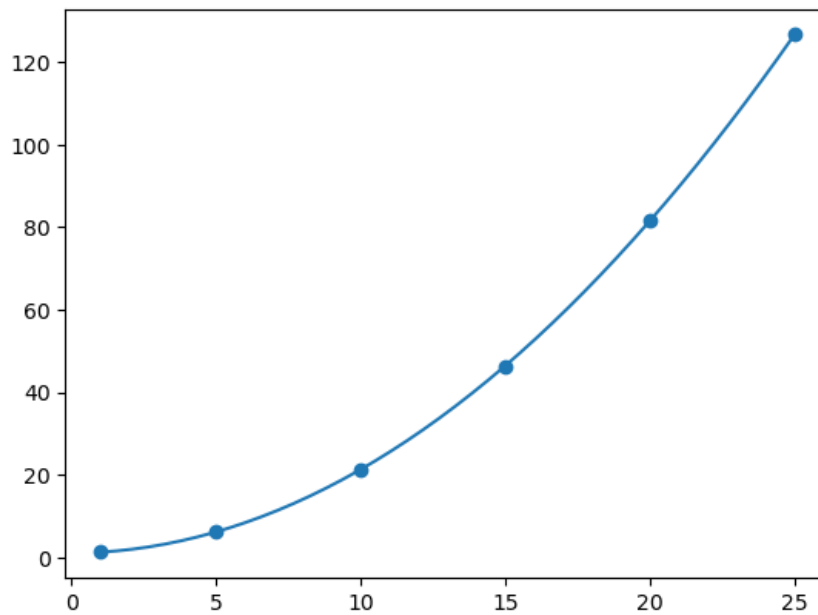
Amb aquest conjunt de dades podem intuir que potser un model que s'ajusta a les nostres dades seria un model de regressió lineal simple.





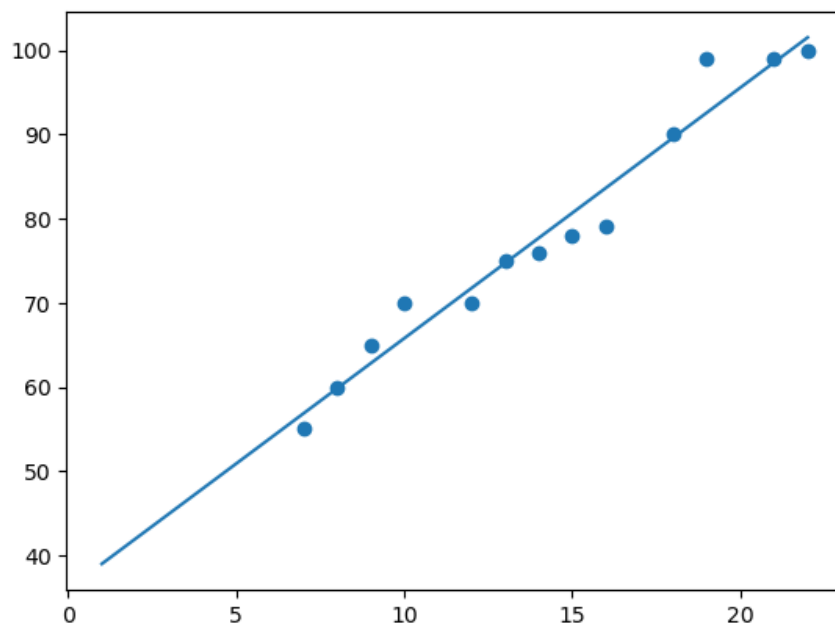
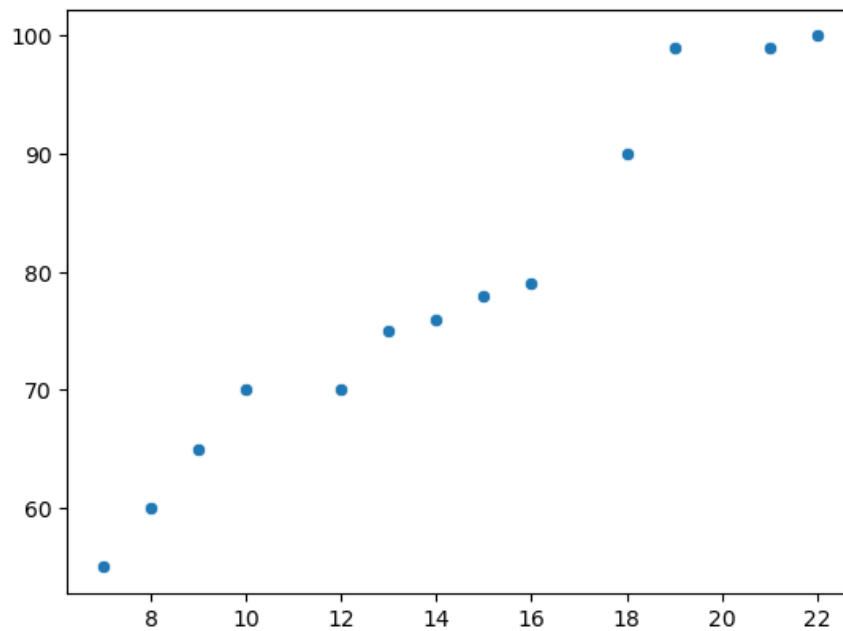
De fet si fem la recta de regressió s'ajusta, però veiem que hi ha alguns punts que estan per sobre i alguns punts que estan per sota.

En canvi si hi apliquem una corba aquesta sembla que s'ajusta molt millor a les nostres dades.

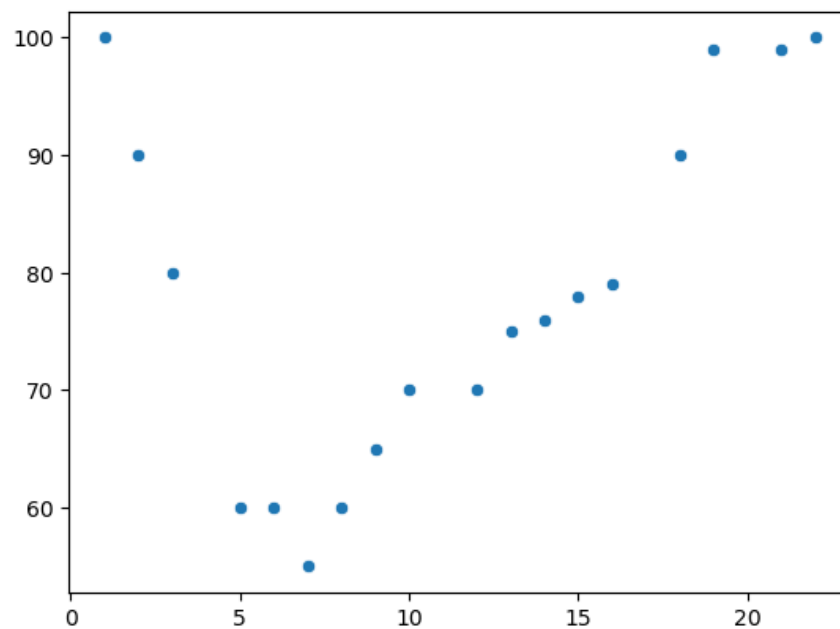


Exemple 2

En aquest segon exemple podem intuir que una recta de regressió lineal simple ens modelarà bé el model

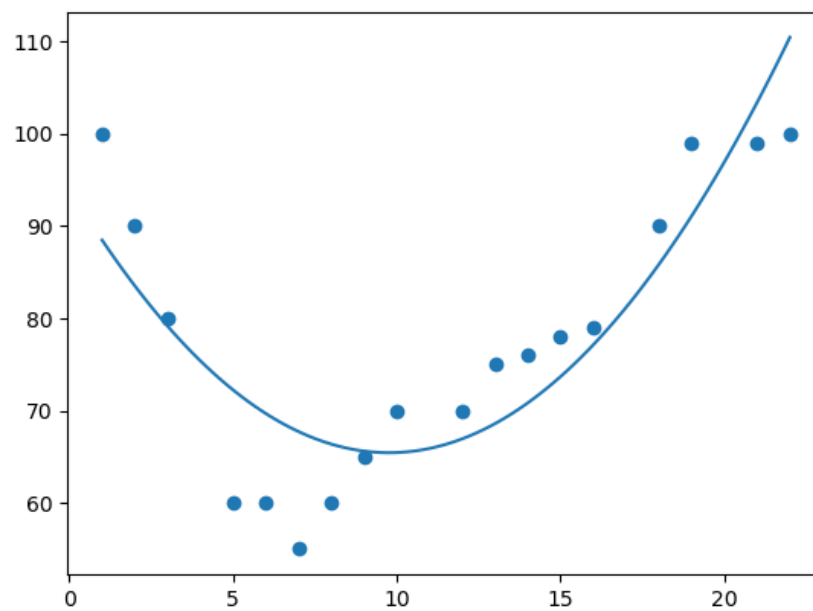


Però resulta que ens donen algunes dades més i veiem el següent mapa de punts.

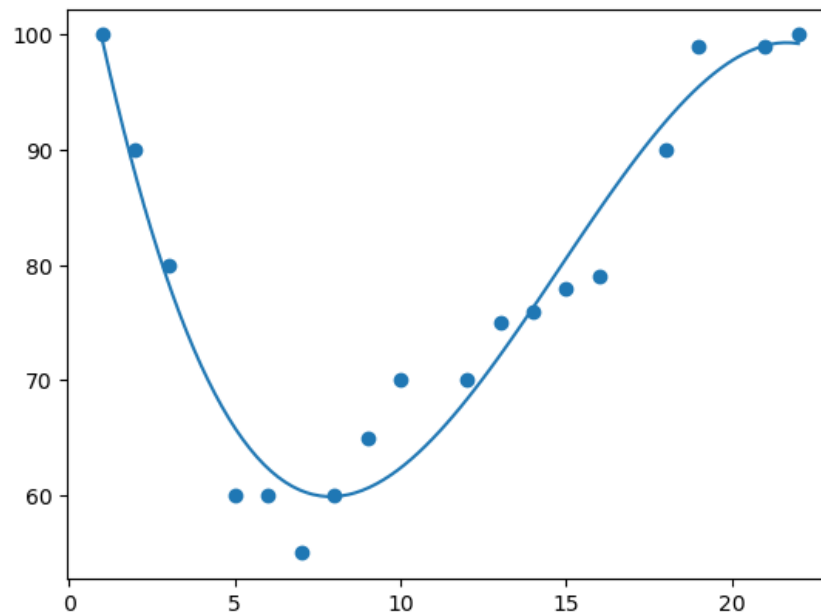


En aquí ja veiem que amb una recta no podrem representar els punts i és aquí a on entraria una regressió lineal polinòmica.

Quan ho provem amb una corba (polinomi de grau 2) tampoc veiem una aproximació al model



Però en canvi, si ho provem amb un polinomi de grau 3 si que veiem que la línia representa força bé les nostres dades.



Avantatges d'ús de la regressió polinòmica

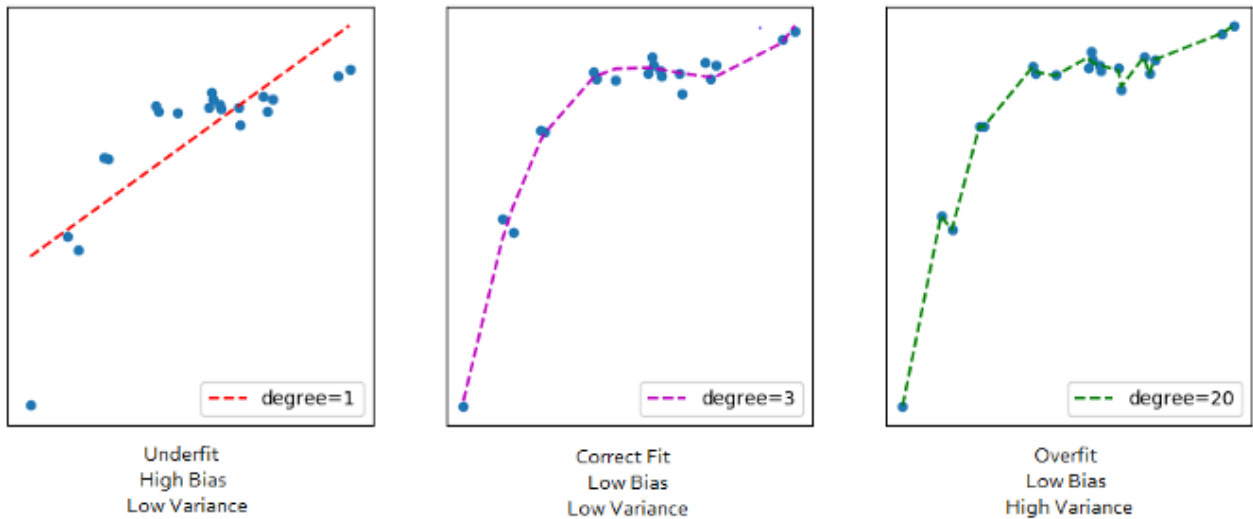
- Com s'ha dit anteriorment podem modelar relacions no lineals entre variables
- Augmentem la varietat de funcions diferents que podem utilitzar per ajustar el model.
- En definitiva, és una eina flexible que es pot utilitzar per adaptar-se a una gran varietat de distribucions de punts de dades.

Problemes de la regressió polinòmica

Encara que podem pensar que la regressió polinòmica ens pot solucionar molts problemes, no és una eina universal i desafortunadament també té un bon nombre de problemes:

- Els models de regressió polinòmica són propensos a sobre ajustar-se. A mesura que augmentem el grau del polinomi per obtenir menys error podem tenir un sobre ajust que ens pot portar a un overfitting ràpidament.
- Hem de focalitzar-nos en els extrems de la corba i veure si les formes i/o tendències tenen sentit.

Perquè polinomis de grau elevat poden produir resultats estranys a l'extrapolació.



Exemples

[0220 regressio lineal polinomica](#) : Exemples de la construcció dels gràfics anteriors

[0221 regressio lineal polinomica - pressio by temperatura](#): Exemple de creació d'un model de regressió polinòmic amb Python i Scikit-learn

Models lineals regularitzats

Quan entrenem un model, especialment un model complex (com una regressió amb moltes variables o una xarxa neuronal amb moltes capes), **el model pot aprendre massa bé les dades d'entrenament provocant sobreajustament (*overfitting*)**.

Altres problemes que ens podem trobar amb la regressió lineal:

- La regressió lineal es veurà perjudicada per variables independents correlacionades.
- La incorporació de totes les variables independents pot ser que no aportin informació rellevant. Això a part de no aportar informació al model, complica la seva interpretació.

La **regularització** introdueix un mecanisme que **penalitza la complexitat del model** per evitar aquests problemes.

La regularització consisteix a **afegir un terme de penalització** a la funció de cost o pèrdua (*loss function*) del model.

Mitjançant la regularització forcem a que els coeficients de les variables independents tendeixin a zero, minimitzant el risc d'*overfitting*, reduint la variància, atenuant l'efecte de la correlació entre predictors i reduint la influència en el model dels predictors menys significatius.

Si en un model de regressió, per exemple, el que intentem de minimitzar és l'error quadràtic mig MSE (funció de pèrdua L2) a on penalitzem els valors de paràmetres massa grans.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Amb la regularització afegim un terme addicional:

$$MSE_{reg} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \cdot P(\beta)$$

On:

- λ és el paràmetre de regularització, que controla la intensitat de la penalització.
- $P(\beta)$ és la funció de penalització, que pot variar segons el tipus de regularització

Regularització L1 (Lasso Regression)

La regressió Least Absolute Shrinkage and Selection Operator (Lasso) és una versió de regularització que **penalitz**a els valors absoluts dels coeficients:

$$P(\beta) = \sum_{j=1}^n |\beta_j|$$

Això fa que alguns coeficients **es converteixin exactament en zero**, fent que el model elimini variables no rellevants.

Aquesta versió de regularització **ajuda a seleccionar els variables més importants** ja que podríem dir que estira els coeficients petits cap a zero.

Quan la fem servir:

- Tens moltes variables, però sospites que només unes poques són realment útils.
- Necessites fer selecció automàtica de característiques (*feature selection*).
- Vols obtenir un model més simple i interpretable.

Per **exemple** volem predir el rendiment d'un estudiant a partir de 50 variables (hores d'estudi, son, estrès, dietes, ús del mòbil, etc.), però sospites que només 5 o 6 són rellevants.

Compte !!

- Pot eliminar massa variables si α és alt o si hi ha correlació entre variables.

Quina diferència hi pot haver respecte la selecció de variable mitjançant Forward do Backward selection?

Són mètodes estadístics clàssics basats en regressió lineal tradicional (OLS) i criteris estadístics com: p-value dels coeficients, ..

Avantatges dels mètodes de selecció:

- Interpretació clara i clàssica (basada en significació estadística).
- Bona per conjunts petits o models interpretables.
- Ideal per entendre *quines variables realment expliquen Y*

Inconvenients dels mètodes de selecció:

- Pot ser computacionalment lent amb moltes variables.

- **No té en compte la multicolinearitat** → pot deixar variables redundants.
- Pot fer *overfitting* igualment (triari el model que millor s'adapta a l'entrenament però no generalitza).
- Petits canvis a les dades poden canviar el model final.

Per exemple el mètode de *Backward* o *Forward* (p-value) va provant ingredients un a un, tastant la sopa i traient l'ingredient que no aporta sabor significatiu (manual, estadístic, lent). En canvi el Lasso Regressions és com si tinguessis un robot que ajusta automàticament a quantitat de cada ingredient i elimina els que no aporten sabor (automàtic, computacional, modern).

Regularització L2 (Ridge *Regression*)

La regressió Least Absolute Shrinkage and Selection Operator (Lasso) és una versió de regularització que penalitza els quadrats dels coeficients.

$$P(\beta) = \sum_{j=1}^n \beta_j^2$$
$$MSE_{reg} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \cdot P(\beta)$$

Això fa que els coeficients grans es redueixin gradualment, però no arriben a ser exactament zero.

És útil quan totes les variables tenen una mica d'importància.

És útil per reduir la variància del model (quan canvien les prediccions si canviem una mica les dades d'entrenament) o quan tenim moltes variables correlacionades

Direm que el model:

- Té alta variància si el model és molt “sensible” a petites variacions en les dades
- Té baixa variància si el model és estable i canvia poc.

Quan la fem servir:

- Tens **moltes variables** i sospites que **algunes estan correlacionades**.
- **No vols eliminar** variables, només **reduir el seu pes**.
- Vols evitar que alguns coeficients es disparin i provoquin **sobreajustament**.

Per **exemple** volem predir la demanda d'energia d'una ciutat amb moltes variables (temperatura, hora del dia, humitat, dia de la setmana, etc.), on algunes estan correlacionades.

Regularització xarxa elàstica (*Elastic Net*) – L1 +L2

La regularització *Elastic Net* combina L1 i L2

$$P(\beta) = r \sum_{j=1}^n |\beta_j| + (1 - r) \sum_{j=1}^n \beta_j^2$$

Aquest mètode de regularització aprofita el millor de cada tipus:

- L1 → redueix a zero alguns coeficients.
- L2 → manté l'estabilitat del model.

En aquest cas el terme de regularització és la suma ponderada dels termes de regularització de Ridge i Lasso i es pot controlar la relació mitjançant r .

- Quan $r = 0$, la xarxa elàstica és equivalent a la regressió Lasso (L1)
- Quan $r = 1$, la xarxa elàstica és equivalent a la regressió Ridge (L2)

Resum

Model	Penalització	Efecte	Elimina variables?	Ús ideal
Linear Regression	✗ Cap	Sense control de complexitat	✗ No	Models simples i interpretables
Ridge (L2)	Quadrats dels coeficients	Redueix coeficients grans	✗ No	Quan hi ha correlacions i moltes variables útils
Lasso (L1)	Valors absoluts dels coeficients	Força coeficients a zero	☑ Sí	Per fer selecció de variables
Elastic Net (L1 + L2)	Combinació	Equilibra L1 i L2	☑ Sí (parcialment)	Quan hi ha moltes variables i correlacions

Imaginem que estàs entrenant un model com si fos un **gimnasta**:

- **LinearRegression:** pot estirar-se tant com vulgui per ajustar-se a cada punt. Risc d'“esquinçar-se”

(sobreajustament).

- **Ridge (L2):** porta una **faixa elàstica** que li limita els moviments extrems. Es mou prou, però amb control.
- **Lasso (L1):** a més de la faixa, té un entrenador que li diu: “Si aquest múscul no serveix, no l’activis” → elimina variables inútils.

Elastic Net: combina les dues estratègies.

Recursos:

- Tutorial “Step-by-Step Guide to Implementing Elastic Net Regression in Python”: https://ujangriswanto08.medium.com/step-by-step-guide-to-implementing-elastic-net-regression-in-python-eff1757aad0a?utm_source=chatgpt.com
- Kaggle Notebook “Multicollinearity / Ridge / Lasso / ElasticNet Regression”: https://www.kaggle.com/code/pranjalpandey12/multicollinearity-ridge-lasso-elasticnet-regression?utm_source=chatgpt.com
- Analytics Vidhya — “Ridge and Lasso Regression in Python” (Tutorial complet amb exemples i interpretació de resultats): https://www.analyticsvidhya.com/blog/2016/01/ridge-lasso-regression-python-complete-tutorial/?utm_source=chatgpt.com
- Car prediction: <https://www.kaggle.com/code/mohaiminul101/car-price-prediction>

Configuració hiperparàmetre lambda (λ)

Tots aquests 3 models tenen un hiperparàmetre que controla la força de la regularització, i trobar el millor valor d'aquest hiperparàmetre és clau per obtenir el millor rendiment del model.

Què passa si lambda (λ) és massa gran o massa petit?

- **λ Molt petit ($\rightarrow 0$):** Penalització gairebé nul·la $\rightarrow$ model similar a una regressió lineal normal. Pot sobreajustar.
- **λ Molt gran:** Penalització molt forta $\rightarrow$ els coeficients s'acosten a 0 $\rightarrow$ el model es torna massa simple (infraajustat).

Com trobem el millor valor per hiperparàmetre?

Una manera de fer-ho és mitjançant fent una cerca amb validació creuada (*SearchCV*) Scikit-learn té versions dels models **Ridge** i **Lasso** que ja fan la validació creuada internament.

```
from sklearn.linear_model import RidgeCV, LassoCV
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_diabetes

# Dades d'exemple
X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Ridge amb validació creuada
ridge_cv = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5)
ridge_cv.fit(X_train, y_train)
print("Millor alpha Ridge:", ridge_cv.alpha_)

# Lasso amb validació creuada
lasso_cv = LassoCV(alphas=[0.001, 0.01, 0.1, 1, 10], cv=5, random_state=42)
lasso_cv.fit(X_train, y_train)
print("Millor alpha Lasso:", lasso_cv.alpha_)
```

alphas $\rightarrow$ conjunt de valors que volem provar.

cv=5 $\rightarrow$ fa validació creuada de 5 particions.

El model escull l'alpha que dona el menor error mitjà de validació.

Resultat: `ridge_cv.alpha_` i `lasso_cv.alpha_` t'indiquen el millor valor automàticament

Per ElasticNetCV

Funciona igual, però també ajusta el `l1_ratio`, que és la proporció entre L1 i L2.

```
from sklearn.linear_model import ElasticNetCV

elastic_cv = ElasticNetCV(
    alphas=[0.001, 0.01, 0.1, 1, 10],
    l1_ratio=[0.1, 0.5, 0.9],
    cv=5,
    random_state=42
)
elastic_cv.fit(X_train, y_train)

print("Millor alpha:", elastic_cv.alpha_)
print("Millor l1_ratio:", elastic_cv.l1_ratio_)
```

GridSearchCV

En existeix `GridSearchCV` que és capaç de buscar el millor hiperparàmetre de qualsevol model (no només regressions).

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge

params = {"alpha": [0.01, 0.1, 1, 10, 100]}
ridge = Ridge()

grid = GridSearchCV(ridge, params, cv=5, scoring="neg_mean_squared_error")
grid.fit(X_train, y_train)

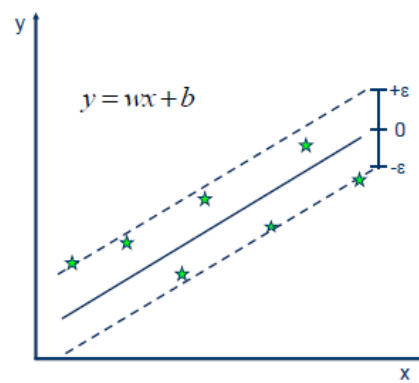
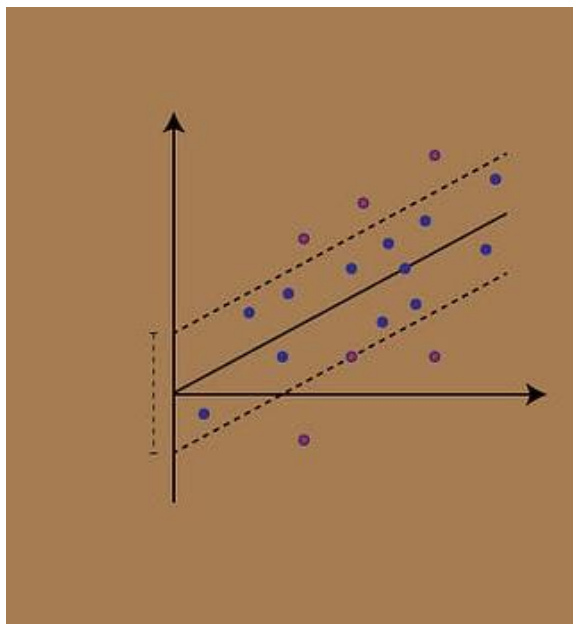
print("Millor alpha:", grid.best_params_["alpha"])
print("Millor score:", -grid.best_score_)
```

Aquest a vegades és útil per quan ens trobem amb dades que tenen un ordre temporal (eries temporals) a on la validació creuada ha de realitzar subgrups ordenant les observacions. Això s'aconsegueix mitjançant `sklearn.model_selection.TimeSeriesSplit` i `GridSearchCV`. [ML scikit learn](#)

Suport Vector Machines

Com se sap, en la regressió lineal tracem una línia recta a través dels punts de dades, i després projectem verticalment aquests punts sobre la línia per trobar la distància mínima.

Farem una cosa similar, però en lloc de dibuixar només la línia de regressió, **dibuixarem una mena de tub** (línies discontinúes de les imatges). No ens preocuparem pels punts que estiguin dins del tub o just a sobre de les línies discontinúes. Ara bé, pels punts que estiguin fora del tub, projectarem la seva projecció vertical sobre les línies discontinúes, no sobre l'eix central del tub. Això reduirà els errors i obtindrem les distàncies mínimes.



- Solution:
$$\min \frac{1}{2} \|w\|^2$$
- Constraints:
$$y_i - wx_i - b \leq \epsilon$$
$$wx_i + b - y_i \leq \epsilon$$

Aquest mètode s'anomena vector de suport perquè els punts que es troben fora del tub són els que anomenem vectors.

Decision Trees (Regressió)

Arbres de decisió

Un arbre de decisió és un model que pren decisions seguint una **estructura jeràrquica en forma d'arbre**. Cada **node intern** representa una **pregunta/prova sobre un atribut** ("condició binària sobre una variable"), **cada branca** correspon a una **resposta possible** de la pregunta/prova i **cada fulla representa una possible** (sí/no o major/menor que un valor).

El camí que segueix una observació de l'arrel fins a una fulla (node terminal) **determina la seva predicció**.

Els mètodes estadístics i de *machine learning* basats en arbres formen part d'un conjunt de tècniques supervisades no paramètriques que divideixen l'espai dels predictors en **regions senzilles i ben definides**, on resulta molt més fàcil analitzar i comprendre les relacions entre les variables. Aquesta capacitat per **segmentar i simplificar espais complexos** és, de fet, el que els atorga **gran part del seu poder i flexibilitat** en la modelització predictiva.

L'estructura de l'arbre es construeix seleccionant de manera recursiva els atributs que divideixen de manera òptima l'espai de dades.

Avantatges:

- **Fàcil interpretació:** Són models intuïtius i comprensibles fins i tot quan les relacions entre variables són complexes.
- **Tractament mixt de variables:** Poden gestionar variables numèriques i categòriques (encara que depèn de la implementació de la llibreria).
- **Sense supòsits estadístics:** En ser mètodes no paramètrics, no requereixen que les dades segueixin una distribució concreta.
- **Poc preprocés necessari:** No cal normalitzar ni estandarditzar les dades i sovint requereixen menys neteja prèvia.
- **Robustos a valors atípics:** No es veuen gaire afectats per valors anòmals.
- **Versatilitat:** Es poden aplicar tant a problemes de **classificació** com de **regressió**.

Desavantatges:

- **Baixa capacitat predictiva d'un sol arbre:** Són models amb alta variància i tendència al sobreajustament (overfitting). Per això es prefereix combinar-ne molts (com a Random Forest o Boosting).
- **Sensibles al desbalanceig de classes:** Si una classe domina sobre les altres, el model pot aprendre un patró esbiaixat.
- **Pèrdua d'informació en variables contínues:** Quan divideixen valors continus, aquests es converteixen en intervals i es perd part de la seva precisió.

Els arbres de decisió els trobem en problemes de regressió i en problemes de classificació. Seguidament explicarem els arbres de decisió en un context de regressió.

Arbres de regressió

Els arbres de regressió són una variant dels arbres de decisió utilitzats quan la variable resposta és numèrica i contínua (com el preu d'un producte, la temperatura o el temps d'entrega d'una comanda).

En un arbre de regressió, cada observació "viaja" per l'arbre seguint una sèrie de preguntes binàries del tipus: "El valor de la variable X és més gran que 50?", "El client té més de 3 anys d'antiguitat?"

En funció de les respostes, l'observació baixa per una branca o una altra fins a arribar a un node terminal (fulla).

La predicció final per aquesta observació és la mitjana dels valors reals (Y) de les mostres d'entrenament que van caure en el mateix node.

Exemple: Imaginem que som un assessor immobiliari que volem donar el preu d'un habitatge. Podem fer el seguit de preguntes:

"El pis té més de 100 m²?" → Sí "Està al centre?" → Sí "Té garatge?"

Segons les respostes, acaba recomanant un preu mitjà basat en pisos similars.

Això és exactament el que fa un arbre de regressió.

L'objectiu del model és dividir l'espai dels predictor en **regions senzilles, no solapades**, dins de les quals

les observacions siguin e més homogènies possible.

Aquest procés es fa en dues etapes principals:

1. Dividir l'espai dels predictors en regions $J(R_1, R_2, \dots, R_J)$. Encara que teòricament les regions podrien tenir qualsevol forma, es limiten a formes rectangulars (de múltiples dimensions), perquè això simplifica el càlcul i facilita la interpretació.
2. Assignar un valor de predicció a cada regió, que normalment és la mitjana de Y dins d'aquella regió.

L'objectiu final és trobar aquelles divisions (tant en quins predictors com en quins valors de tall) que minimitzin l'error de predicció.

En un arbre de regressió, el criteri més utilitzat per trobar la millor divisió és la **suma de quadrats residuals (RSS, Residual Sum of Squares)**:

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

On:

- J conjunt de regions $(R_1, R_2, \dots, R_J)$
- R_j és una de les regions creades.
- $\hat{y}_{R_j}$ és la mitjana dels valors Y dins d'aquella regió.

Busquem dividir les dades de manera que les observacions dins de cada regió siguin el més semblants possible (és a dir, que el RSS sigui mínim).

Com que és impossible provar totes les divisions possibles s'utilitza un mètode voraç anomenat **recursive binary splitting** (divisió binària recursiva). Aquesta solució segueix la mateixa idea de la selecció de variables stepwise (*backward* o *forward*) en una regressió lineal multivariable.

Algorisme Recursive Binary Splitting

- **Comença amb totes les dades juntes.**

Totes les observacions formen part d'una única regió inicial.

- **Per cada predictor X_j** , es proven tots els punts de tall possibles (s).
 - Si el predictor és numèric, s'ordenen els valors i es consideren punts intermedis entre valors consecutius.
 - Si és categòric, es proven combinacions de categories (nivells).
- **Es calcula el RSS** resultant de dividir les dades en dos grups:
 - $R_1 = \{X \mid X_j < s\}$
 - $R_2 = \{X \mid X_j \geq s\}$
- **S'escull el predictor X_j i el punt de tall s** que redueix més el RSS total.

Aquesta és la millor divisió en aquell moment.
- **Es repeteix el procés recursivament** en cada nova regió creada (R_1 i R_2), **fins** a arribar a una **condició d'aturada**, com ara:
 - Profunditat màxima de l'arbre.
 - Nombre mínim d'observacions per node.
 - Millora mínima en el RSS després d'una nova divisió.

Cada divisió es decideix amb la informació disponible en aquell punt, sense considerar com afectarà les divisions futures. Això pot provocar que l'arbre no sigui globalment òptim, però és una bona aproximació pràctica.

Un arbre massa profund pot adaptar-se massa a les dades d'entrenament (overfitting).

Per evitar-ho, s'utilitza una tècnica anomenada poda (pruning), que consisteix a tallar algunes branques de l'arbre després de construir-lo, mantenint només les divisions que milloren realment la capacitat predictiva.

Scikit-learn permet controlar-ho amb paràmetres com:

- `max_depth`: profunditat màxima de l'arbre (controla overfitting).
- `min_samples_split`: mínim de mostres per fer una nova divisió.
- `min_samples_leaf`: mínim de mostres a cada fulla.

- `criterion`: criteri d'avaluació ("squared_error", "friedman_mse", "absolute_error").

Per evitar l'overfitting podem:

- limitar `max_depth`
- Incrementar `min_samples_leaf` o `min_samples_split`

Podem utilitzar **Cross Validation** mitjançant **GridSearchCV** per trobar la millor configuració de paràmetres

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "max_depth": [2, 3, 4, 5, 6],
    "min_samples_leaf": [1, 2, 5]
}

grid = GridSearchCV(DecisionTreeRegressor(random_state=42),
                    param_grid, cv=5, scoring='r2')
grid.fit(X_train, y_train)

print("Millor paràmetre:", grid.best_params_)
print("Millor R2:", grid.best_score_)
```

Predicció amb un arbre de regressió

Quan el model està entrenat.

- Una nova observació "viatge" per l'arbre responent les preguntes fins arribar a un node terminal/fulla.
- La predicció és la mitjana dels valors Y de les observacions d'entrenament que estan en aquest node terminal.

També es podrien utilitzar altres estadístics, com la mediana o quantils, si el problema ho requereix (per exemple, si hi ha molts outliers).

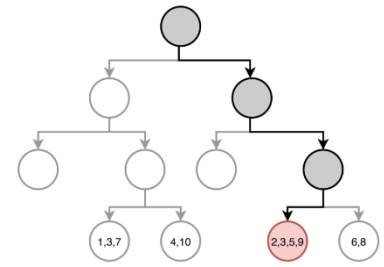
Exemple

Suposem que disposem de 10 observacions, cada una amb un valor de la variable depenent Y i una variable depenent X

id	1	2	3	4	5	6	7	8	9	10
Y	10	18	24	8	2	9	16	10	20	14
X	...	...	...	...	...	...	...	...	...	...

Donada una nova observació utilitzariem l'arbre de decisió fins arribar a un node fulla. En aquest node terminal es detallarà l'índex de les observacions de l'entrenament.

El valor que predirem serà la mitjana de la variable resposta Y de les observacions del node fulla.



En l'exemple que ens ocupa:

$$\hat{\mu} = \frac{18+24+2+20}{4} = 16$$

Els arbres de regressió són **models senzills, interpretables i potents** per capturar relacions no lineals entre variables.

Encara que per si sols poden ser inestables, la seva simplicitat els converteix en **la base fonamental de molts models avançats**, com **Random Forest** o **XGBoost (Gradient Boosting)**, que milloren la seva precisió combinant-ne molts.

Models Combinats (*Ensemble Models*)

Moltes vegades em sentit a parlar de la intel·ligència col·lectiva a on un conjunt de persones aleatòries resolen una pregunta complexa com pot ser quantes boles de xiclet hi ha en un bol. Si ponderem el resultat de milers de persones veurem que el resultat s'apropa a la solució i molt probablement la resposta és millor que la d'un expert.



Els models combinats (*ensemble*) són conjunts de **múltiples models que treballen conjuntament** per millorar el rendiment i reduir el sobreajustament (overfitting).

La idea clau darrere de ***l'ensembling*** és entrenar diversos models sobre diferents subconjunts de dades i després agregar-ne les prediccions:

- En tasques de **classificació**, la predicció final s'obté mitjançant **votació majoritària** (la classe més freqüent entre les diferents prediccions).
- En problemes de **regressió**, la predicció final és la **mitjana** de les prediccions de tots els models (per exemple, arbres).

Estratègies d'*Ensemble Learning*

Les dues estratègies més comunes per construir *ensemble models* són **Bagging** i **Boosting**

Bagging (Bootstrap Aggregating)

En català podríem traduir-lo com a agrupament amb mostres aleatòries. Ja que consisteixen en fer diverses mostres (*bootstrap*) i després agrupar (*aggregate*) els models resultats.

La idea és crear múltiples models entrenant-los sobre subconjunts aleatoris (amb reemplaçament) de les dades d'entrenament.

Cada model **s'entrena** de manera **independent**.

Per exemple, el Random Forest utilitza *bagging* entrenant diversos arbres de decisió sobre subconjunts diferents del conjunt de dades i fent la mitjana de les seves prediccions.

Una de les grans **avantatges** del *Bagging* és que **redueix la variància** i **ajuda a prevenir l'overfitting**.

Boosting (Potenciació seqüencial)

En català la podríem traduir per Potenciació seqüencial.

Els models s'entrenen **seqüencialment**, i cada nou model se centra a corregir els errors dels models anteriors.

El *boosting* **assigna més pes** a les instàncies classificades incorrectament, fent que el model següent se centri en aquests casos més difícils.

A diferència del *bagging*, els models **no s'entrenen de manera independent**.

Al final, tots els models contribueixen a la predicció final mitjançant **vot ponderat o agregació**.

Exemples de mètodes de Boosting:

- AdaBoost (Adaptive Boosting): Augmenta el pes de les mostres classificades incorrectament perquè el següent model se centri en elles.
- Gradient Boosting: Utilitza el gradient descent per minimitzar l'error de manera iterativa.
- XGBoost (Extreme Gradient Boosting): Una versió optimitzada del Gradient Boosting, coneguda per la seva eficiència i precisió. Fa servir arbres de decisió potenciats pel gradient (GBDTs), una versió altament optimitzada dels CART (Classification and Regression Trees):
 - Cerca de divisions basada en histogrames per augmentar la velocitat.
 - Estratègia de poda (arbres poc profunds) per evitar el sobreajustament.

L'avantatge del Boosting és que redueix el biaix (quan es devien les prediccions d'un model amb respecte als valors reals) i millora la precisió, però pot ser propens al sobre ajustament si no es regularitza bé.

Stacking (Generalització Apliada)

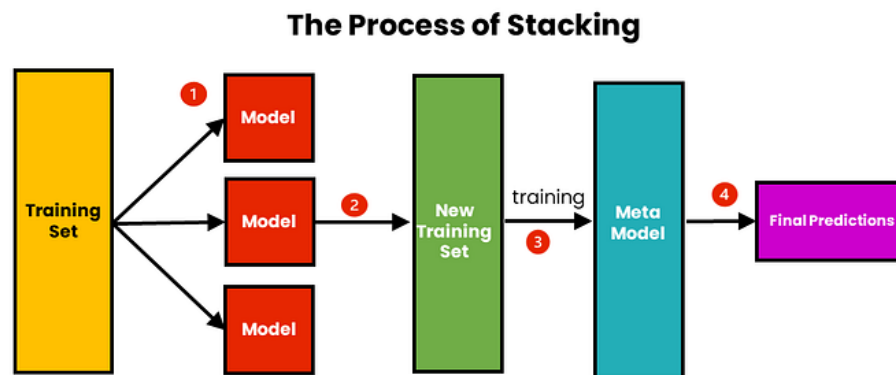
Stacking és una tècnica que combina **diversos models diferents** (no necessàriament del mateix tipus) per construir un meta model més potent i equilibrat.

A diferència del *Bagging* i el *Boosting*, que combinen **molts models iguals** (normalment arbres), el *Stacking* utilitza **models diversos** (*heterogenis*) i aprèn a **compondre les seves prediccions**.

L'Stacking és l'estratègia d'ensemble més flexible i potent per a regressió quan disposem de models diversos (lineals, arbres, SVM, etc.).

El seu metamodel aprèn a ponderar les fortaleces de cada model base, aconseguint sovint un rendiment superior al de qualsevol model individual o fins i tot al Random Forest o Boosting per separat.

Per exemple tenim 3 models predictors diferents per predir un valor amb el resultat (3.1, 2.7 i 2.9). Aquests 3 valors predits es passen a un altre model anomenat *blender* o metamodel que pren aquestes prediccions com a entrades i realitza la predicció final (3.0)



Imatge de Google

Article interessant: https://medium.com/@brijesh_soni/stacking-to-improve-model-performance-a-comprehensive-guide-on-ensemble-learning-in-python-9ed53c93ce28

Random Forest for Regression

Un Random Forest (bosc aleatori) és un model basat en la combinació de molts arbres de decisió.

L'objectiu és reduir els inconvenients d'un sol arbre (com el sobreajustament o alta variància) i aconseguir un model més estable i precís.

Cada arbre **s'entrena independentment** amb una **mostra aleatòria** de les dades originals, obtinguda mitjançant una tècnica anomenada **bootstrapping**.

- Cada arbre veu un **subconjunt lleugerament diferent** de les dades d'entrenament.
- **Algunes observacions poden repetir-se**, i d'altres potser no apareixen en absolut en aquell arbre concret.

En el cas de regressió cada arbre genera una predicció numèrica i el model final calcula la **mitjana** d'aquestes prediccions.

Com aconseguir que aquest model redueixi la variància?

- **Bootstrapping (mostres diferents):** Cada arbre s'entrena amb un subconjunt aleatori de dades. Això fa que cada arbre aprengui un patró lleugerament diferent i **cometi errors independents**. Quan s'agrupen totes les prediccions, aquests **errors es compensen entre si**, reduint la variància global.
- **Selecció aleatòria de variables:** Quan un arbre busca el millor punt de tall per dividir un node, **no considera totes les variables** disponibles, sinó **només un subconjunt aleatori**. Això evita que tots els arbres acabin sent iguals i promou la **diversitat** dins el bosc.

Exemple: Imaginem que volem predir el preu d'una casa amb diferents arbres de decisió.

- Si entrenem un únic arbre molt profund, aprendrà exactament tots els preus del conjunt d'entrenament. → R^2 del train = 0.99, però en test baixa a 0.60 → overfitting!
- Si creem 100 arbres diferents (amb subconjunts de dades i variables), cada arbre s'equivocarà d'una manera diferent.

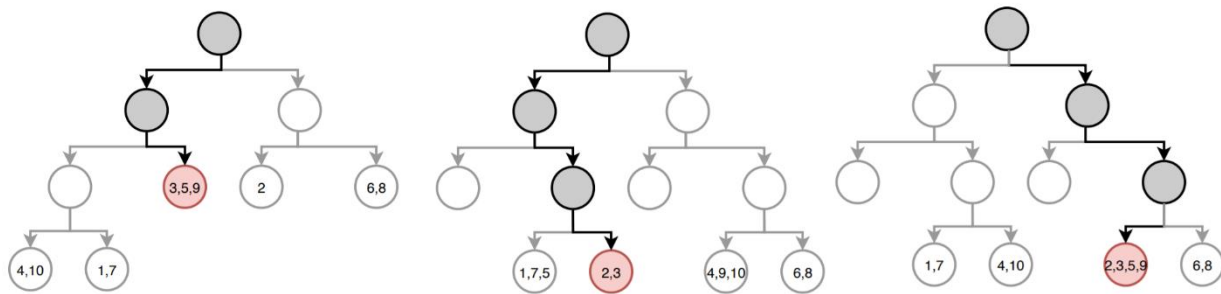
Però quan fas la mitjana de totes les seves prediccions, el soroll es redueix: → R^2 del train $\approx$ 0.92 i del test $\approx$ 0.90 → model estable i generalitzable!

És com escoltar l'opinió de 100 experts que han vist dades diferents.

Cadascun pot equivocar-se una mica, però si agregues totes les seves opinions, el resultat global sol ser molt més fiable que el d'un sol expert.

Si agafem l'exemple de l'arbre de decisió:

id	1	2	3	4	5	6	7	8	9	10
Y	10	18	24	8	2	9	16	10	20	14
X	...	...	...	...	...	...	...	...	...	...



Imatge extreta de cienciadedatos.net

El valor de cada arbre, tal i com hem vist anteriorment en els arbres de decisió és la mitjana aritmètica del valor de la variable dependent Y .

$$\hat{y}_{arbre1} = \frac{24+2+20}{3} = 15.333$$

$$\hat{y}_{arbre2} = \frac{18+24}{2} = 21$$

$$\hat{y}_{arbre3} = \frac{18+24+2+20}{4} = 16$$

La predicció final del model és la mitjana de totes les prediccions dels arbres individuals:

$$\hat{\mu} = \frac{15.333+21+16}{3} = 17.4$$

Out-of-Bag

El valor **Out-of-Bag** (OOB) és una d'aquelles joies amagades del Random Forest. Serveix per avaluar el model sense haver de reservar un conjunt de validació extern i, ben entès, és una eina molt útil per ajustar hiperparàmetres ràpidament.

Què és i com es calcula l'OOB en un Random Forest Regressor?

Si recordem cada arbre **no** s'entrena amb totes les mostres, sinó amb una mostra bootstrap: es tria aleatòriament (amb reemplaçament) el mateix nombre de files que té el dataset, però algunes es repeteixen i algunes **no surten**.

Les que **no han sortit** per entrenar aquell arbre s'anomenen **mostres "out-of-bag"** per aquell arbre.

Per tant per cada observació del dataset, hi haurà algun subconjunt d'arbres que no l'han vist mai en

l'entrenament. Això vol dir que podem utilitzar aquesta observació com una dada de validació/test per aquests arbres.

Per calcular aquest oob hem d'activar el paràmetre `oob_score=True` en la classe `RandomForestRegressor` de Scikit-Learn.

Com es calcula:

1. Actives `oob_score=True` al `RandomForestRegressor`.
2. El bosc s'entrena normalment.
3. Després de l'entrenament, per a **cada mostra** del train:
 - a. es miren **tots els arbres que no la van usar per entrenar-se** (els arbres per als quals és "OOB")
 - b. aquests arbres fan una **predicció** per aquella mostra
 - c. es fa la **mitjana** d'aquestes prediccions → aquesta és la **predicció OOB** per a aquella mostra
4. Quan tenim la predicció OOB per a **totes** les mostres, es calcula una mètrica de rendiment (en el cas de sklearn és r^2).

Ens pot servir per comparar diferents hiperparàmetres i fer *tuning* del nostre model: `n_estimators`, `max_features`, `max_depth`... sense estar dividint dades contínuament.

⚠ L'Out-of-Bang és una estimació i no pot ser mai un substitut absolut del test. Si tenim Test avaluem el Test.

Exemple Scikit-Learn

A Scikit-Learn la classe per entrenar un Random Forest és `RandomForestRegressor`. Aquesta classe té els següents paràmetres per defecte:

- `n_estimators=100`
- `criterion='squared_error'`
- `max_depth=None`
- `min_samples_split=2`
- `min_samples_leaf=1`
- `min_weight_fraction_leaf=0.0`
- `max_features=1.0`
- `max_leaf_nodes=None`
- `min_impurity_decrease=0.0`
- `bootstrap=True`
- `oob_score=False`

- `n_jobs=None`
- `random_state=None`
- `verbose=0`
- `warm_start=False`
- `ccp_alpha=0.0`
- `max_samples=None`

```

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import root_mean_squared_error
from sklearn.metrics import r2_score

# Load the Boston Housing dataset
boston = fetch_openml(name='boston')
df = boston['data']
df['MEDV'] = boston['target']

# Dividim el conjunt de dades train/test
X_train, X_test, y_train, y_test = train_test_split(
    df.drop(columns="MEDV"),
    df['MEDV'],
    test_size = 0.25,
    random_state = 24
)

# Creem i entrenem el model
model_rfr = RandomForestRegressor(random_state = 54)
model_rfr.fit(X_train, y_train)

y_pred = model_rfr.predict(X=X_test)

# Avaluem el model
rmse = root_mean_squared_error(y_true=y_test, y_pred=y_pred)
print(f"El error (rmse) de test es: {rmse}")

r2= r2_score(y_true=y_test, y_pred=y_pred)
print(f"El error (r2) de test es: {r2}")

```

Extra-Trees

Quan estem fent créixer un arbre en un RandomForest, a cada node només considerem un subconjunt aleatori de característiques a la divisió, però és possible augmentar més l'aleatorietat **utilitzant també llinars aleatoris** per **cada característica**, en comptes de buscar els millors llinars possibles (tal com fan els arbres de decisió).

Per fer això només cal configurar `splitter="random"` alhora de crear la classe `DecisionTreeRegression`.

Un bosc d'arbres "extremadament" aleatoris s'anomena *Extremely Randomized Trees*, abreviant *Extra-Trees*.

A banda d'utilitzar boscos a on tenim **més aleatorietat** i no haver de buscar els llinars òptims per cada característica fa que els **Extra-Trees s'entrenin en molta més rapidesa que els Random Forest**.

En un *Random Forest*, cada arbre "pensa molt" per trobar el millor punt de tall (la millor pregunta). En canvi, en un *Extra Trees*, cada arbre diu: "no cal que pensi tant, faré un tall aleatori i ja veurem què passa".

Això pot semblar caòtic, però quan n'agrupes centenars, el **soroll individual es cancel·la** i la mitjana resulta sorprenentment precisa.

Això converteix *Extra Trees* en un model:

- molt **ràpid d'entrenar**, perquè no calcula tants possibles punts de tall;
- amb **baixa variància**;
- i amb **bon rendiment** en dades grans o sorolloses.

Amb Scikit-Learn podem crear *ExtraTrees* mitjançant la classe `ExtraTreesRegressor`. La signatura de la classe és idèntica a la de `RandomForestRegressor`, però té el paràmetre `bootstrap=False`.

Algú podria pensar, Per què tenim `bootstrap=False` amb un Extra Tree?

Perquè com s'ha dit en lloc de buscar "l millor punt de tall" com fa el Random Forest, Extra Trees escull de manera completament aleatòria el punt de tall dins del rang de valors possibles sense optimitzar el MSE de forma exhaustiva. Per tant, cada arbre és molt diferent dels altres encara que s'entreni amb les mateixes dades.

A vegades pot interessar `bootstrap=True`, ja que tindràs **dues fonts d'aleatorietat** (dades i punts de tall), podràs calcular un **OOB score** com amb el Random Forest, però el model pot **trigar més a entrenar-se** i **guanyar poc o res en precisió**.

A la pràctica, normalment s'ha vist que activar el *bootstrap* en Extra Trees no millora significativament el resultat i pot empitjorar-la amb conjunts petits.

Gradient Boosting Regressor

Com s'ha comentat anteriorment els models de boosting són aquells que s'entrenen **seqüencialment**, i cada nou model se centra a corregir els errors dels models anteriors.

És un dels algorismes més potents i utilitzats en Machine Learning per a dades tabulars (taules amb files i columnes, com les d'un Excel o base de dades).

En el cas de Gradient Boosting està format per molts arbres de decisió petits que s'entrenen un darrere l'altre, de manera que cada nou arbre intenta corregir els errors anteriors.

La predicció d'una nova observació s'obté combinant les prediccions de tots els arbres individuals que formen el model.

Avantatges

- **Potència predictiva molt alta:** Combina molts arbres simples que junts aconsegueixen un model molt precís.
- **Aprèn de manera iterativa:** Cada arbre corregeix els errors dels anteriors, fent el model cada cop millor.
- **Flexible:** Funciona tant per regressió com per classificació.
- **No necessita gaire preprocesament:** Els arbres gestionen variables numèriques i categòriques, valors absents o *outliers*.
- **Identifica variables importants:** El model pot indicar quins predictors són més rellevants.

Desavantatges i/o limitacions

- **Més lent d'entrenar:** Com que els arbres s'entrenen seqüencialment, pot trigar més que un Random Forest (que pot entrenar en paral·lel).
- **Risc de sobreajustament (overfitting):** Si afegim massa arbres o el *learning rate* és massa alt, el model pot memoritzar el conjunt d'entrenament.

- **Difícil d'interpretar:** Un sol arbre és fàcil d'explicar, però centenars d'arbres combinats no tant.
- **No extrapola bé:** Com tots els models basats en arbres, no sap predir fora del rang dels valors entrenat

És un model que requereix de One-Hot Encoding per tractar les variables categòriques.

Passos

- **Pas 1. Es comença amb un model simple:** Normalment es parteix d'un model molt senzill, com un petit arbre de decisió (també anomenat *stump*), que fa una predicció inicial. En un problema de regressió, aquesta predicció inicial sol ser simplement la mitjana dels valors de sortida.

$$f_1(x)$$

- **Pas 2. Es calculen els errors (residus):** Es calcula quant s'ha equivocat el model per a cada observació.

Aquests errors indiquen on el model no ha sabut capturar bé la relació entre les variables.

$$r_1 = y - f_1(x)$$

- **Pas 3. S'entrena un nou arbre sobre els residus:** Ara entrenem un segon arbre que no intenta predir la variable objectiu original, sinó els errors del primer model. Això vol dir que el nou arbre aprèn on i com s'ha equivocat el primer.

$$f_2(x) \approx r_1 = y - f_1(x)$$

- **Pas 4. S'actualitzen les prediccions:** El nou arbre fa les seves pròpies prediccions i aquestes s'afegeixen a les anteriors amb un pes petit (el *learning rate* (λ) o taxa d'aprenentatge). Així, el model va corregint a si mateix de forma suau i progressiva.

$$F_2(x) = f_1(x) + \lambda \cdot f_2(x)$$

- **Pas 5. Es repeteix el procés:** Es van afegint més arbres, cadascun aprenent dels residus del model anterior, fins que:
 - Els errors són molt petits, o
 - Arribem al nombre màxim d'arbres configurats.

Al final, la predicció final és la suma ponderada de tots els arbres construïts.

$$F_m(x) = F_0 + \eta \sum_{i=1}^m f_i(x)$$

On:

- F_0 és la predicció inicial (la mitjana de y),
- $f_i(x)$ és la predicció de l'arbre número i ,
- η és el *learning rate*,
- i $F_m(x)$ és la predicció final després de sumar totes les contribucions.

Per tant, per fer una **nova predicció**, només cal aplicar aquest mateix procés sobre el nou valor de X .

Regularització i evitar l'overfitting

Com que cada nou arbre s'adapta als residus dels anteriors, el model pot arribar a sobre ajustar-se (overfitting), és a dir, aprendre massa bé els detalls del conjunt d'entrenament i perdre capacitat de generalitzar.

Aquí és on entra el paràmetre **learning rate (λ)**. Normalment els valors van de **0.01 a 0.1**. Si es configura amb un valor baix, per exemple 0.05, necessitarem més arbres en el conjunt (*ensemble*) per ajustar el conjunt d'entrenament, però normalment, les prediccions generalitzaran millor. Si afegim molts arbres podem patir sobre ajust.

Binning

Un dels punts més costosos del Gradient Boosting és trobar el punt de tall (threshold) òptim per dividir les dades en cada node de tots els arbres. Això quan tens moltes files i moltes columnes, és el que més temps li fa perdre.

El *binning* intenta de solucionar aquest coll d'ampolla. Tots els GBM moderns fan servir binning.

La idea és posar "bins", cubells en una variable numèrica continua en un conjunt de valors discrets que representen un interval.

En comptes de treballar amb 3.42, 3.47, 3.51, 3.99, 4.02, 4.11, 4.13, ...

Treballarem amb: bin_1, bin_1, bin_2, bin_3, bin_3, ...

És adir no treballem amb **tots** els valors possibles, sinó amb uns quants **punts representatius** (els límits dels *bins*).

Això és molt semblant al que fas quan fem un **histograma**: no dibuixem una barra per cada valor, sinó per

cada interval.

Com es fa el binning

Hi ha diverses maners, però la més típica en GBM és:

- L'algorisme recorre una variable i mira la seva distribució.
- La divideix en , per exemple, 256 bins (aquest número és configurable).
- Aquests bins es solen fer per quantils: cada bin té més o menys el mateix número d'observacions. Per què quantils? Perquè si tens una variable molt concentrada (p. ex. salaris entre 1.000 i 1.500) i uns quants valors molt grans (20.000), el binning per quantils fa que hi hagi bins "on hi ha dades" i no 200 bins buits.

Fase d'entrenament

Quan l'algorisme vol decidir "on tallo aquesta variable?", ja no ha de mirar milers de valors.

Només ha de provar aquests 256 punts (els límits de cada bin).

És una cerca **molt més curta**.

A més, com que els bins són enters petits (0,1,2,...255), es poden guardar en arrays molt compactes i fer càlculs molt ràpids

Early Stopping

Quan entrenem un model de Gradient Boosting, cada iteració (o arbre) millora una mica el model.

Al principi les millores són grans, però a **mesura que afegim més arbres**, el model **aprèn detalls cada cop més petits del conjunt d'entrenament**.

Arriba un punt en què **el model deixa d'aprendre patrons generals** i comença a **aprendre soroll o casos particulars del train** i cau en overfitting.

L'Early Stopping és una tècnica per aturar automàticament l'entrenament quan el model deixa de millorar realment.

Com que cada arbre corregeix els errors del anteriors i amb prou arbres es pot arribar a reproduir **perfectament** les dades de train.

1. El que es fa és dividir les dades en train i validació
2. S'entrena el model iterativament afegint arbres d'un en un.
3. A cada iteració es calcula una mètrica de rendiment (accuracy, R2, logloss o RMSE) sobre el conjunt de validació.
4. Comparem la millor mètric obtinguda fins al moment amb l'actual:
 - Si hi ha millora guardem aquest model com el millor
 - Si no hi ha millora durant "diverses iteracions seguides" aturem l'entrenament.

Hiperparàmetres que controlen Early Stopping

- `validation_fraction = 0.1` # proporció del train que es reserva per validació
- `n_iter_no_change = 10` # nombre d'iteracions sense millora abans d'aturar
- `tol = 0.001` # mínima millora requerida per considerar que hi ha millora

El parèmtre `n_iter_no_change` per ser una bon alternativa molt pràctica per fer una cerca exhaustiva o fent moltes proves amb molts paràmetres mitjançant `GridSearchCV` o `RandomizedSerarchCV`

```
from sklearn.ensemble import GradientBoostingRegressor

model = GradientBoostingRegressor(
    n_estimators=1000,          # nombre màxim d'arbres possibles
    learning_rate=0.05,
    validation_fraction=0.1,
    n_iter_no_change=10,
    tol=0.001,
    random_state=42
)
model.fit(X_train, y_train)
```

XGBoost i LightGBM

GradientBoosting és la versió clàssica implementada al biblioteca Sckit-learn. És ideal per a finalitats didàctiques o projectes petits, però avui dia s'utilitza menys en entorns a gran escala perquè van aparèixer millores.

Aquest models ofereixen altres característiques addicionals, incloent **l'acceleració de la GPU** entre altres.

LightGBM

LightGBM és una implementació desenvolupada per Microsoft i optimitzada per a **velocitat i eficiència en memòria**.

Fa servir tècniques innovadores com el **Leaf-wise Growth** (creixement per fulles) i **histogram-based learning**.

✿ Característiques:

- **Molt ràpid** en dades grans
- Fa **binning**
- **Admet valors missing**
- Realitza **entrenament paral·lel·litzat** (multicore i distributed learning)
- Compatible amb GPU
- **Tracta variables categòriques de forma nativa** (no necessita One-Hot Encoding)

💡 Casos d'ús. Quan necessites **velocitat i gran escala**, com per exemple:

- Predicció de demanda en temps real.
- Rànquing de productes en e-commerce.
- Classificació de frauds amb grans volums de dades.

🏠 Hiperparàmetres clau

És l'algorisme que més exposa el control de binning perquè la seva arquitectura interna està basada en histogrames

Hiperparàmetre	Descripció
<code>max_bin</code>	Número màxim de bins per a cada característica. Per defecte és 255. Si el poses més alt, el model té més precisió però entrenarà més lent i consumirà més memòria
<code>min_data_in_bin</code>	Nombre mínim d'exemples que ha de tenir cada bin. Evita bins

	buits o massa petits.
<code>bin_construct_sample_cnt</code>	Nombre de mostres usades per construir els bins (si el dataset és molt gran). Serveix per fer un binning aproximat en lloc de calcular-lo sobre totes les dades.
<code>feature_pre_filter</code>	Si és <code>True</code> , LightGBM pot eliminar automàticament variables amb pocs bins vàlids abans d'entrenar (ajuda a la velocitat).

```
import lightgbm as lgb

params = {
    'objective': 'regression',
    'learning_rate': 0.05,
    'num_leaves': 31,
    'max_bin': 255, # controla la resolució del binning
    'min_data_in_bin': 5
}

train_data = lgb.Dataset(X_train, label=y_train)
model = lgb.train(params, train_data, num_boost_round=300)
```

XGBoost Regressor

XGBoost és una de les implementacions més populars i robustes. Va revolucionar les competicions de *machine learning* per la seva gran capacitat d'optimització i la seva eficiència.

Característiques:

- Permet valors missing
- Pot fer binning o no
- Entrenament paral·lelitzat i optimitzat en memòria.
- Suport per GPU.
- Admet variables categòriques (des de versions recents).
- Ofereix gran control d'hiperparàmetres i regularització L1/L2, reduint el risc d'overfitting.

💡 **Casos d'ús:** Model preferit **projectes empresarials exigents:**

- Predicció de risc de crèdit.
- Models de retenció de clients.
- Predicció de preus o vendes amb alta precisió.

🔧 **Hiperparàmetres clau**

Hiperparàmetre	Descripció
<code>max_bin</code>	Número màxim de bins per a cada característica. Per defecte és 256. Si el poses més alt, el model té més precisió però entrenarà més lent i consumirà més memòria
<code>tree_method</code>	Si el poses com "hist" o "gpu_hist", actives l'ús de binning automàtic. Altres modes com "exact" no fan binning (i són molt més lents).
<code>grow_policy</code>	"depthwise" (nivell per nivell) o "lossguide" (com LightGBM, fulla a fulla). No afecta el binning però influeix en com s'utilitza

Altres models Gradient Boosting

- HistGradientBoosting (sklearn)
- CatBoost
- H2O.ai

Article respecte el Gradient Boosting:

https://cienciadedatos.net/documentos/py09_gradient_boosting_python

Resum Ensemble Models

Model	Combinació d'arbres	Forma d'entrenar	Tolerància a soroll	Risc d'overfitting	Paràmetres clau
Decision Tree	1 arbre	Independent	Alta	Alt	max_depth
Random Forest	Molts arbres independents	En paral·lel (vot majoritari)	Alta	Baix	n_estimators, max_features
Gradient Boosting	Molts arbres seqüencials	Seqüencial (aprèn dels errors)	Mitjana	Mitjà-alt	learning_rate, n_estimators, max_depth, n_iter_no_change
XGBoost / LightGBM / CatBoost	Gradient Boosting optimitzat	Seqüencial + optimitzat	Alta	Controlable	molt optimitzats

Stacking - Exemple

Un altre opció d'ensemble model és apilar models

Resum General de models de Regressió

Situació real	Model que triaria primer	Per què
Vull entendre els efectes (coeficients) i tot sembla recte	Regressió Lineal	Simple, interpretable, baseline obligat
Hi ha colinearitats (features semblants)	Ridge o Elastic Net	Estabilitza coeficients, generalitza millor
Tinc moltes variables i vull un model curt	Lasso o Elastic Net	El Lasso apaga coeficients (selecció)
Les relacions són corbes però suaus i	SVR (RBF)	Captura no-lineal suaument amb

tinc poques dades		pocs punts
Vull un baseline fort no-lineal sense massa tuning	Random Forest	Robúst, poc sensible a hiperparàmetres
Necessito màxima precisió en tabular	Gradient Boosting / XGBoost / LGBM	Són els “champions” si es regulen bé
Necessito explicacions amb regles	Decision Tree (o RuleFit)	Regles if-else, fàcil d’explicar

Models Supervisats de Classificació

En els diferents models de classificació tenim:

- Models lineal: Regressió Logística
- Models basats en arbres: DecisionTreeClassifier, RandomForestClassifier, XGBoost, LightGM, CatBoost
- Models probabilístics: Naive Bayes, GaussianNB
- Models neuronals:MLPClassifier (sckikit-learn),Xarxes neuronals (KerasPyTorch), softmax a la capa de sortida.

Avaluar els models de classificació

La matriu de confusió és una de les mètriques més intuïtives i fàcils que s'utilitzen per avaluar un model de classificació.

En **matriu de confusió** tenim: els Verdaders positius (**TP** – *True Positives*), els Verdaders negatius (**TN** – *True Negatives*), els Falsos positius (**FP** – *False Positives*) i els Falsos negatius (**FN** – *False Negatives*)
Per exemple si ens trobem al davant d'un model capaç de predir si una persona té càncer o no.

Les files representen les classes reals i les columnes representen la classe predita.

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN	FP
	POSITIU	FN	TP

Accuracy (Exactitud)

Aquesta mètrica és bàsica i indica el número d'elements classificats correctament en comparació amb el número total d'elements que tenim. Ens diu com de bo en general és la nostra classificació

$$accuracy = \frac{TP + TN}{TP + FP + FN + TN(total)} = \frac{P + N}{P + N}$$

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN	FP
	POSITIU	FN	TP

Compte !! : Encara que sigui un bon punt de partida no és bo refiar-nos només d'aquesta mètrica, perquè si estem al davant de dades molt desequilibrades pot donar falses expectatives.

Per exemple, imaginem que tenim un volum de dades de 100 persones i només 5 persones tenen càncer i tenim un model que l'*accuracy* ens surt del 96%. Podríem pensar que és un model molt bo, però no és així.

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN(95)	FP(0)
	POSITIU	FN(4)	TP(1)

$$accuracy = \frac{1 + 95}{100} = 96\%$$

Encara que aquest model sigui totalment ineficaç per predir si una persona té càncer o no, el valor de l'exactitud és molt alta.



- Quan les classes estan ben equilibrades (50%-50%, 60%-40%...).
- Quan l'impacte dels errors FP i FN és similar.



- Quan hi ha desequilibri de classes (per exemple, frau 1% vs 99%).
- Un model pot dir sempre "no té càncer" i tenir 95% d'accuracy.

Precision (Precisió)

Aquesta mètrica representa el número de de veritables positius (TP) que són realment positius en comparació amb el número total de valors positius predits.

Estem avaluant les dades del model segons el desenvolupament de les prediccions “positives”. Dit d’una altra manera de tots els casos que diem “positiu”, quants ho són realment.

També podríem dir que mesura la **fiabilitat d’un positiu del model**

$$precision = \frac{TP}{TP + FP}$$

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN	FP
	POSITIU	FN	TP

En l’exemple anterior veiem que tenim un *precision* del 100% i ens podem pensar que estem davant d’un bon model, però no és així!

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN(95)	FP(0)
	POSITIU	FN(4)	TP(1)

$$precision = \frac{1}{1 + 0} = 1 = 100\%$$

Per exemple mirem si el nostre model é capaç de detectar si un correu és SPAM o no i tenim els següents valors:

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN(0)	FP(95)
	POSITIU	FN(0)	TP(5)

$$precision = \frac{5}{5 + 95} = 0.05 = 5\%$$

Aquest 5% ens fa una idea que el nostre model no encerta gaire els positius i a més envia molts del nostres correu a SPAM.



- Podríem utilitzar-la **quan els falsos positius (FP) siguin cars**. Un model que tingui una baixa precisió i el cost de FP és car. Seria catastròfic.

Recall (Sensibilitat / Exhaustivitat)

La mètrica de recall mostra la quantitat de veritables positius que el model ha classificat en funció del nombre total de valors positius.

Dels casos reals positius, quants en prediem correctament. Mesura **com d'exhaustiu és el model detectant positius reals**.

$$recall = \frac{TP}{TP + FN}$$

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN	FP
	POSITIU	FN	TP

Si mirem el nostre primer exemple

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN(95)	FP(0)
	POSITIU	FN(4)	TP(1)

$$recall = \frac{1}{5} = 0.2 = 20\%$$

Aquesta mètrica ens diu que n'enganyem de positius només un 20%!



- L'utilitzarem quan els **falsos negatius (FN) són greus**. Per exemple si hem de detectar càncer o no. No detectar-lo és crític).

Specificity (especificitat)

Mostra la quantitat de **verdaders negatius** que el model ha classificat **respecte el total de negatius**

Es calcula com el número de prediccions negatives correctes dividint per el número total de negatius. Els casos reals de "No", Quants n'hem predit correctament.

Seria el cas contrari a la sensibilitat/recall

$$specificity = \frac{TN}{TN + FP}$$

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN	FP
	POSITIU	FN	TP

Si mirem el nostre primer exemple

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN(95)	FP(0)
	POSITIU	FN(4)	TP(1)

$$specificity = \frac{95}{95} = 1 = 100\%$$

Aquesta mètrica ja ens diu que no estem enganxant cap de negatiu real!

F1 Score

Si ens fixem en què volem tenir una mètrica que es centri amb els positius i intenti d'agafar un compromís entre la precisió i el *recall*.

Això és el que fa la mètrica de F1. Aquesta mètrica permet comparar dos classificadors. El F1 és la mitjana harmònica de la precisió i el *recall*. Es a dir dona més pes als valors baixos.

La millor puntuació F1 és 1 i la pitjor un 0

$$F1 = \frac{2}{\frac{1}{precision} + \frac{1}{recall}} = 2 \cdot \frac{precision \cdot recall}{precision + recall} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

El valor de F1 prefereix als classificadors que tinguin un precision i un recall similars.

Si mirem el nostre primer exemple

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN(95)	FP(0)
	POSITIU	FN(4)	TP(1)

$$F1 = 2 \cdot \frac{1 \cdot 0.2}{1 + 0.2} = 2 \cdot \frac{0.2}{1.2} = 0.33$$

Model molt dolent amb un 0.33

Resum

No sempre hem de mirar el F1 Score. En alguns contextos, ens interessa més la precisió, mentre que en d'altres ens interessa més el recall.

Per exemple si estem davant d'un **classificador** per **detectar lladres en imatges** de vigilància i tenim un **30% de precisió**, però un **99% de recall** significarà que algunes vegades tractarem com a positius lladres que no ho seran, per tant els vigilants hauran d'actuar, però per altra banda atraparem tots els lladres.

		PREDICCIÓ		
		NEGATIU	POSITIU	
REALITAT	NEGATIU	TN)	FP	
	POSITIU	FN	TP	99%
			33%	

Per desgràcia, no podem tenir ambdues coses precisió i recall. Ja que si augmentem la precisió reduïm el recall i viceversa.

Compensació precisió/recall

Tot model de classificació utilitza un mecanisme per classificar les classes. Podem moure aquest percentatge de manera vertical dins la matriu de confusió per tal de millorar els nostres falsos positius (FP), però cal tenir en compte que això també empitjorem els nostres True Positive (TP)

		PREDICCIÓ	
		NEGATIU	POSITIU
REALITAT	NEGATIU	TN	FP
	POSITIU	FN	TP

Podem moure aquesta línia vertical per intentar millorar els nostres TN o TP, però això també afectarà els nostres FN o FP.

En el cas d'una Regressió logística tenim un llindar de 0.5.

- Si baixem el llindar p.e. $th=0.3$, pugem el TP, però també estem pujant els FP
- Si pugem el llindar p.e. $th=0.7$, pugem el TN, però també estem pujant els FN

Resum

Llindar	FP	FN	Ús
Alt (0.7-0.9)	Baixos	Alts	Quan és molt perillós donar un fals positiu (spam,...)
Mig 0.5	Normal	Normal	Casos generals
Baix (0.2-0.3)	Alts	Baixos	Quan vols detectar tots els positius (malalties, frau,...)

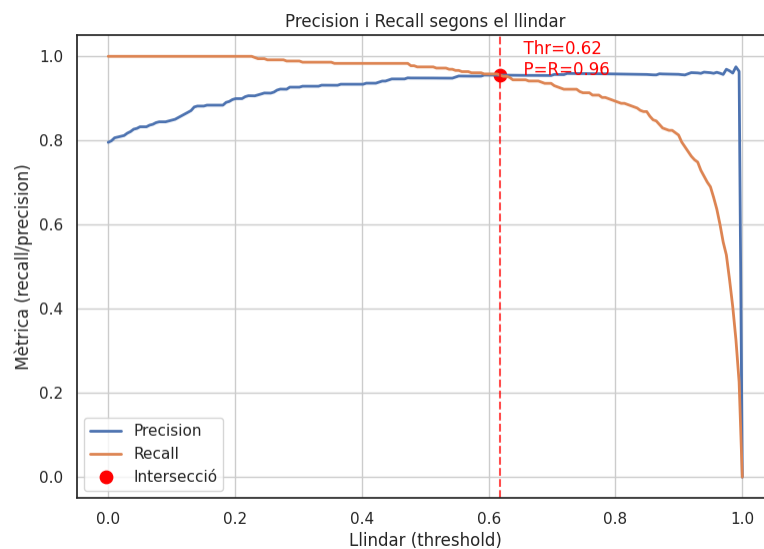
Podem intentar de buscar un compromís entre la precisió i recall.

```
# --- Càlcul de les mètriques segons llindar ---
thresholds = np.linspace(0.0, 1.0, 200)
precisions, recalls = []

for t in thresholds:
    y_pred_t = (y_prob >= t).astype(int)
    precisions.append(precision_score(y_test, y_pred_t, zero_division=0))
    recalls.append(recall_score(y_test, y_pred_t))

# --- Punt mínim entre Precision i Recall) ---
idx_best_thr = np.argmin(np.abs(np.array(precisions) - np.array(recalls)))

best_thr = thresholds[idx_best_thr]
```



Una altra manera de fer-ho seria maximitzant el `f1_score` mitjançant `cross_validation` + `precision_recall_curve`

```
# Probabilitats out-of-fold (no entrenades sobre elles mateixes)
y_prob_cv = cross_val_predict(model, X, y, cv=5, method="predict_proba")[:,1]

precisions, recalls, thresholds = precision_recall_curve(y, y_prob_cv)
f1_scores = 2 * (precisions * recalls) / (precisions + recalls)

idx = np.argmax(f1_scores)
best_thr_cv = thresholds[idx]
```

Un cop hem decidit un **llindar** no podem fer inferència mitjançant un simple **predict** sinó que hem d'utilitzar **predict_proba()** i **aplicar el llindar**. Veiem un exemple utilitzant *LogisticRegression*

```
from sklearn.linear_model import LogisticRegression
import numpy as np

# 1. Entrenament
model = LogisticRegression()
model.fit(X_train, y_train)

# 2. Obtenció probabilitats al conjunt de validació (o OOF)
y_prob_val = model.predict_proba(X_val)[: ,1]

# 3. Trobar threshold òptim (exemple via F1)
best_thr = 0.37 # Suposem que ho hem trobat abans

# 4. Inferència amb threshold òptim
y_prob_test = model.predict_proba(X_test)[: ,1]
y_pred_test = (y_prob_test >= best_thr).astype(int)
```

Podem empaquetar això en una funció

```
def predict_with_thr(model, X, threshold):
    probs = model.predict_proba(X)[: , 1]
    return (probs >= threshold).astype(int)

y_pred_final = predict_with_thr(model, X_new, best_thr)
```

“La regressió logística no decideix si és 0 o 1: només dóna probabilitats. .predict() posa el llindar a 0.5, però tu n’has trobat un de millor o que s’adapti més a les teves necessitats. Per això cal fer servir .predict_proba() i després aplicar la regla: si prob > threshold ⇒ 1, si no ⇒ 0”

Corva ROC i AUC

La corva ROC (*Reciver Operating Characteristic*) és una eina utilitzada en els classificadors binaris.

Va néixer duran la Segona Gerra Mundial, en l'àmbit de la detecció d'enemics en el radar.

Els operadors de radar havien de decidir si un senyal detectar era un avió enemic real (positiu) o només soroll de fons (negatiu)

Per això necessitaven una eina per mesurar la sensibilitat(recall)del rada (és a dir, quants avions detecta) i mesurar la seva especificitat(quantes vegades es confon amb soroll).

La corva ROC és un gràfic que mostra com es comporta un model binari per a tots els possibles llindars (thresholds).

TPR(True Positive Rate)

$$TPR = recall = \frac{TP}{TP + FN}$$

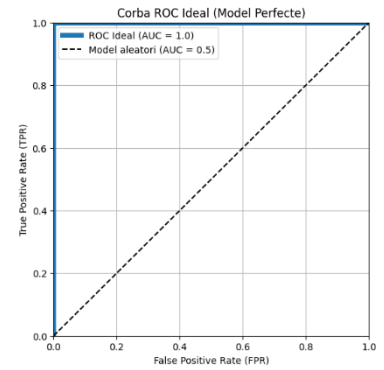
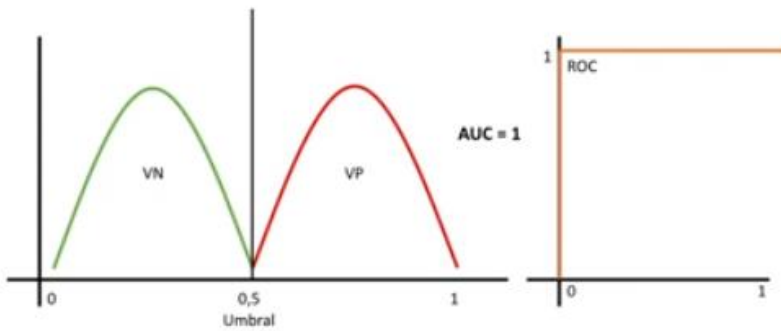
FPR (False Positive Rate)

$$FPR = \frac{FP}{FP + TN} = 1 - especificitat$$

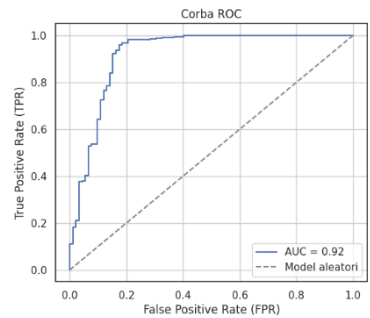
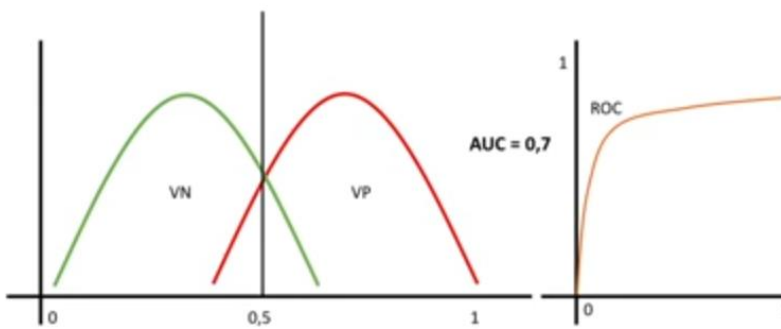
És la proporció de instàncies negatives que s'han classificat de manera incorrecte com a positives. O també anomenada ratio de fals positius.

Per tant la corva ROC és recall vs 1-especificitat

- Un ROC ideal seria TPR=1 amb un FPR=0 en tots els llindars.
- Un ROC mediocre seria TPR=FPR en tots els llindars (diagonal).
- Un ROC terrible TPR < FPR



Un bon classificador és aquell que la corba ROC es manté com a més lluny possible de la diagonal.



AUC

Ja que hem dit que un bon classificador és aquell que la corba ROC es manté com a més lluny possible de la diagonal. És per això que si calculem l'àrea que queda sobre la curva ROC i aquesta és gran podríem dir que estaríem davant d'un bon model. Per tant AUC són les sigles de (Area Under Curve).

AUC no tindrà un valor superior a 1 perquè TPR i FPR mai poden superar a 1. Per tant l'àrea serà com a màxim de 1 ($1 \times 1 = 1$).

AUC tindrà valors que aniran de 0 a 1, però hauríem de considerar 0.5 a 1 perquè 0.5 seria la diagonal. Si AUC tingués un valor entre 0 i 0.5 significaria que el nostre model ordena a l'inrevés.

```

from sklearn.metrics import roc_curve, auc
import numpy as np
import matplotlib.pyplot as plt

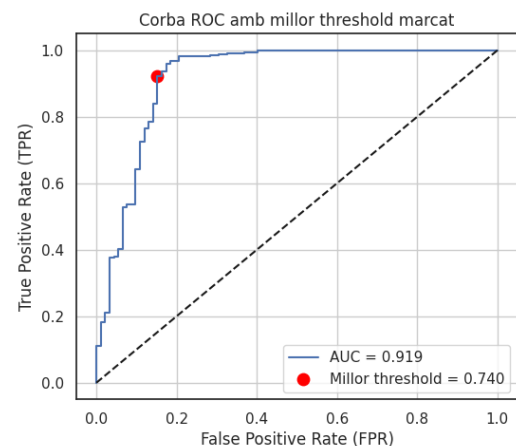
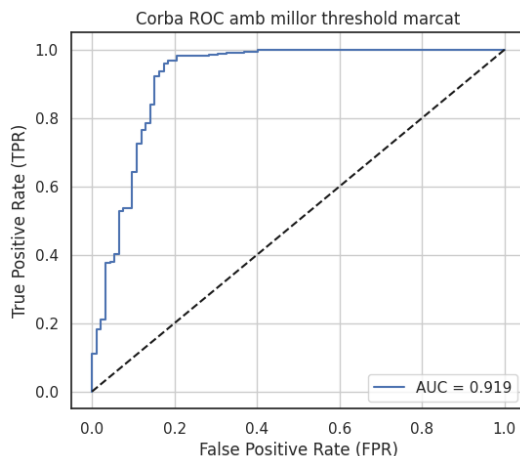
fpr, tpr, thresholds = roc_curve(y_test, y_prob) # Obtenim la corba ROC
roc_auc = auc(fpr, tpr) # AUC

# --- Trobar el millor threshold ---
# Criteri: punt més proper a (0,1), és a dir: # max TPR i minr FPR
distancia = np.sqrt((1 - tpr)**2 + (fpr)**2) # distància euclidiana
best_index = np.argmin(distancia)
best_threshold = thresholds[best_index]

print("Millor threshold segons ROC:", best_threshold)
print("TPR en aquest punt:", tpr[best_index])
print("FPR en aquest punt:", fpr[best_index])
print("AUC:", roc_auc)

# --- Plot ROC ---
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}")
plt.scatter(fpr[best_index], tpr[best_index], color='red', s=80,
            label=f"Millor threshold = {best_threshold:.3f}")
plt.plot([0,1], [0,1], 'k--')
plt.xlabel("False Positive Rate (FPR)")
plt.ylabel("True Positive Rate (TPR)")
plt.title("Corba ROC amb millor threshold marcat")
plt.legend()
plt.grid()
plt.show()

```



Regressió logística

La regressió logística és un model estadístic i de machine learning supervisat dissenyat **per predir una variable categòrica binària**, és a dir, una variable que només pot prendre dos valors possibles (com “sí/no”, “malalt/sà”, “fraudulent/no fraudulent”, “0/1”).

Encara que el seu nom inclou el mot “regressió”, en realitat és un model de classificació, ja que la seva finalitat no és predir un valor numèric sinó una classe.

El que fa és calcular una probabilitat que una observació pertanyi o no a una classe a través d’una combinació lineal de totes les característiques.

Si recordem el model de regressió lineal tenim que intenta ajustar una recta que descrigui la relació entre una variable dependent y i una o més variables independents $x_1, x_2, \dots, x_p \dots$

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

Aquest model és adequat quan y és contínua, però no quan y és binària, ja que :

- Els valors predits poden ser inferiors a 0 o superiors a 1 (no té sentit que ho interpretem com a una probabilitat).
- La relació entre x i la probabilitat de y no és lineal.

Per resoldre això, la regressió logística transforma el resultat de la regressió lineal amb una **funció logística o sigmoide**. Això assegura que la sortida sempre estigui dins l’interval $[0,1]$

Funció sigmoide

La funció sigmoide és una funció matemàtica en forma de “S” que converteix qualsevol valor real $x \in (-\infty, +\infty)$ en un valor entre 0 i 1.

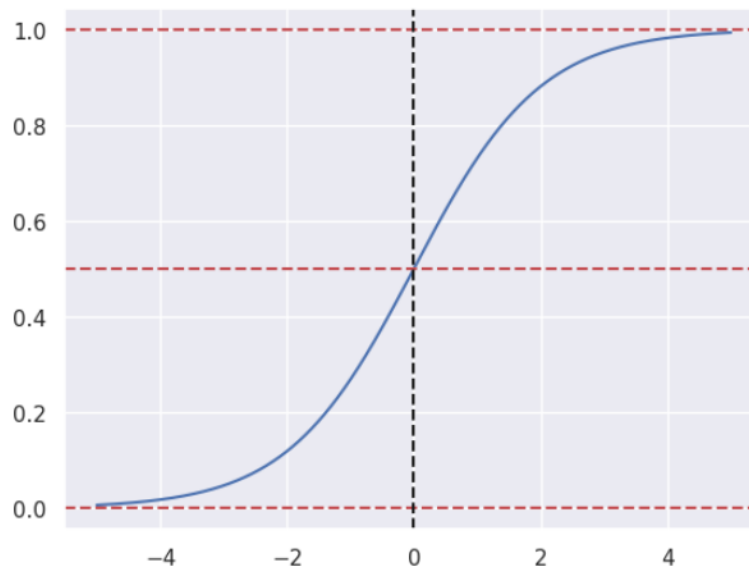
$$f(x) = \frac{1}{1 + e^{-x}}$$

- Per cada $x = -\infty$; $e^{-(-\infty)} = e^{\infty} = \infty \Rightarrow f(-\infty) = \frac{1}{1+\infty} = \frac{1}{\infty} = 0$
- Per cada $x = +\infty$; $e^{-\infty} = e^{-\infty} = \frac{1}{e^{\infty}} = \frac{1}{\infty} = 0 \Rightarrow f(\infty) = \frac{1}{1+0} = \frac{1}{1} = 1$

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-5, 5, 10000)
y = 1 / (1 + np.exp(-x))

plt.plot(x, y)
plt.axhline(0, c = 'r', linestyle = '--')
plt.axhline(0.5, c = 'r', linestyle = '--')
plt.axhline(1, c = 'r', linestyle = '--')
plt.axvline(0, c = 'k', linestyle = '-')
```



Construcció del model

Donat el concepte de funció logística o sigmoide la construcció del model és fàcil d'entendre i el procés es realitza en dues etapes:

Etapa 1

Primer combinem les variables d'entrada amb uns pesos per formar una expressió lineal com la que havíem vist en el cas de regressió lineal:

$$y_{lineal} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \varepsilon$$

Ca cada variable x_i se li assigna un pes β_i que indica **com contribueix** a fer més o menys probable que la sortida sigui 1

Etapa2

Com que el valor de y pot ser qualsevol nombre el transformem en 0,1 mitjançant sigmoide

$$y_{prob} = \text{sigmoide}(y_{lineal}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p)}}$$

Multivariable

De la mateixa manera que podem realitzar regressions lineals multivariables en la regressió logística també ja que el que fem és transformar la funció lineal en una de sigmoide.

Recomanacions

En principi les mateixes premisses d'ús d'un model de regressió lineal el trobem en la regressió logística. Es a dir, hi ha diverses situacions on la regressió logística no és recomanable, no és suficient o directament falla.

- Quan la **relació entre les variables** i la probabilitat **NO és lineal**. Si l'efecte d'una variable té forma de corba, o interaccions o patrons complexes, la regressió logística ho farà malament.
- Quan **les dues variables NO** les podem **separar** mitjanament bé a través d'un **hiperplà**. variables NO són separables ni tan sols aproximadament
- Quan hi ha multicol·linealitat (variables molt correlacionades entre si) tampoc la regressió

logística serà el nostre model.

- Si tenim **moltes característiques** en front de **poques dades**. Com a mínim necessitem 10 instàncies positives per cada coeficient que estimem. Si tenim 300 variables i un dataset petit no funcionarà.
- Si tenim dades molt desequilibrades. Si tenim 1% de la mostra com a classe positiva i un 99% de les mostres a una classe negativa la regressió pot acabar sempre predient la classe majoritària.

En aquest cas la solució passa per utilitzar:

- `LogisticRegression(class_weight='balanced')` balancejar
- Resampling (SMOTE, undersampling)

Naive Bayes

Naive Bayes és un algorisme probabilístic, fàcil d'entendre, rapidíssim i sorprenentment potent. Es pot utilitzar de s de detectar correus SPAM (correu brossa) o analitzar sentiments. Naive Bayes ofereix resultats que rivalitzen amb models complexos, especialment en dades de text.

Naive Bayes és un algorisme de classificació basat en la probabilitat. Sap classificar com un email pot ser SPAM o no SPAM calculant com de probable és que certes característiques (per exemple, paraules d'un correu) pertanyin a cada categoria. Els "toc" ingenu (*naive*) és que assumeix que totes les característiques són independents, una simplificació que el fa ràpid i efectiu, encara que les dades reals gairebé mai compleixin aquesta condició ja que a la realitat trobem que certes característiques estan relacionades unes amb les altres.

Probabilitat

La **probabilitat** és una manera de quantificar la incertesa. Ens diu com d'acceptable és que passi un esdeveniment.

Un valor de probabilitat sempre està entre 0 i 1 encara que de vegades es representi en format de percentatge.

A nivell formal la probabilitat d'un esdeveniment **A** l'escriurem com:

$$P(A)$$

Exemple 1: moneda

- Probabilitat que surti cara : $P(cara) = 0.5$

Exemple 2: dau

- Probabilitat que surti número 1 : $P(1) = 1/6 = 0.16$

En Machine Learning s'utilitza la probabilitat com a freqüència: La probabilitat d'un esdeveniment és la seva freqüència relativa quan s'observa un gran nombre de vegades.

Per exemple si observem que un client compra un productes 200 vegades de 1000 visites:

$$P(compra) \approx \frac{200}{1000} = 0.2$$

Distribució conjunta

La distribució conjunta (*Joint Distribution*) o probabilitat conjunta descriu la probabilitat que **dos o més esdeveniments passin simultàniament (a la vegada)**.

$$P(A, B) \text{ o } P(A \cap B)$$

És dir: “quina és la probabilitat que A i B siguin veritat simultàniament?”.

Considerem dues variables aleatòries, X i Y . La distribució conjunta de X i Y , denotada com $P(X, Y)$ o $f(x, y)$, proporciona les probabilitats associades a cada possible parell de valors (x, y) que X i Y poden prendre. Matemàticament, aquesta probabilitat conjunta es representa com:

$$P(X = x_i, Y = y_j) \text{ o } P(x_i, y_j)$$

Aquí, x_i i y_j representen valors específics que X i Y poden assumir, respectivament.

Propietats:

- **No-negativitat:**

$$P(X = x_i, Y = y_j) \geq 0, \forall (x_i, y_j)$$

- **Normalització:**

$$\sum_{i,j} P(X = x_i, Y = y_j) = 1$$

És a dir, la suma de les probabilitats per a totes les combinacions possibles de valors és igual a 1

El concepte es pot estendre a **més de dues variables**. Per a tres variables (X, Y, Z) , la distribució conjunta es denota com $P(X, Y, Z)$ o $f(x, y, z)$, i descriu les probabilitats de totes les combinacions possibles de valors de X , Y i Z que es produeixen simultàniament.

Exemple. Imaginem que tenim una enquesta de 100 estudiants respecte el sistema operatiu preferit depenent del nivell d'estudis i creem la seva taula de contingència.

Estudis /SO	Windows	Linux	MacOS	Total
Batxillerat	30	5	20	55
Cicles FP	20	40	10	70
Universitat	40	10	25	75
Total	90	55	55	200

Cada cel·la ens dona la probabilitat conjunt de

$$P(X = \text{Estudis}, Y = \text{SO})$$

- $P(\text{CiclesFP}, \text{Windows}) = \frac{20}{200} = 0.1 = 10\%$
- $P(\text{Universitat}, \text{Linux}) = \frac{10}{200} = 0.05 = 5\%$

Aquestes probabilitats són les probabilitats aleatòries que simultàniament tinguin un nivell d'estudis concret i que tinguin la preferència d'un SO determinat.

Distribució marginal

La distribució marginal o la probabilitat marginal és la probabilitat d'un esdeveniment sense importar els altres.

$$P(A) = P(A, B) + P(A, \neg B)$$

És dir: "quina és la probabilitat única i exclusivament de A?"

Aquesta probabilitat la podríem definir com la **suma de la distribució conjunta de $P(A,B)$** i la distribució conjunta de A i el complement de B $P(A, \neg B)$ $\neg B$ significa que "No passa" .

Si volem generalitzar-ho tenim:

$$P(X) = \sum_y P(X, y)$$

Exemple: Marginals de les observacions de la preferència de SO per nivell d'estudis

Estudis /SO	Windows	Linux	MacOS	Total
Batxillerat	30	5	20	55
Cicles FP	20	40	10	70
Universitat	40	10	25	75
Total	90	55	55	200

$$P(\text{Batxillerat}) = \frac{55}{200} = 0.275 = 27.5\%$$

$$P(\text{Windows}) = \frac{90}{200} = 0.45 = 45\%$$

La marginal és la suma de la conjunta:

$$P(\text{Batxillerat}) = P(\text{Windows}, \text{Batxillerat}) + P(\text{Linux}, \text{Batxillerat}) + P(\text{MacOS}, \text{Batxillerat})$$

$$P(\text{Batxillerat}) = \frac{30}{200} + \frac{5}{200} + \frac{20}{200} = 0.275 = 27.5\% \text{ El mateix valor anterior.}$$

Distribució condicional

La probabilitat condicionada. Fixant un esdeveniment quina probabilitat hi ha que passi el segon?. Les condicionals permeten respondre preguntes del tipus:

- Quin SO prefereixen les persones amb Batxillerat?

$$P(A|B) = \frac{P(A, B)}{P(B)} = \frac{P(A \cap B)}{P(B)}$$

Probabilitat que es produeixi A **sabent es produeix B**

$$P(A|B) = \frac{\text{probabilitat conjunta } A, B}{\text{probabilitat marginal } B} = \frac{\text{casos on passen } A \text{ i } B}{\text{casos on passa només } B}$$

Estudis /SO	Windows	Linux	MacOS	Total
Batxillerat	30	5	20	55
Cicles FP	20	40	10	70
Universitat	40	10	25	75
Total	90	55	55	200

Condicional: SO dins de cada nivell d'estudis

- Batxillerat

$$P(\text{Windows}|\text{Batxillerat}) = \frac{30}{55} = 0.5454 = 55\%$$

$$P(\text{Windows}|\text{Batxillerat}) = \frac{\text{probabilitat conjunta } A, B}{\text{probabilitat marginal } B} = \frac{\frac{30}{200}}{\frac{30}{200} + \frac{5}{200} + \frac{20}{200}} = 0.5454 = 55\%$$

D'aquí en podem treu que:

- A Batxillerat clarament es prefereix Windows
- A Cicles el predominant és Linux
- Els universitaris també prefereixen Windows.

Teorema de Bayes

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

On:

- $P(A|B)$: Probabilitat de l'esdeveniment A un cop hem observat B. (**probabilitat posterior**)
- $P(B|A)$: Probabilitat versemblança: probabilitat d'observar B saben que A és cert.
- $P(A)$: Probabilitat **prèvia (prior)**. La nostra creença abans de veure dades addicionals.
- $P(B)$: Probabilitat **marginal** de B. La probabilitat total d'observar B independentment de la causa

Interpretació informal/intuïtiva del teorema de Bayes

1. El que ja sabíem abans: prior $\rightarrow P(A)$
2. El que ens diu la nova evidència: versemblança $\rightarrow P(B|A)$
3. L'escala global del fenomen: marginal/evidència $\rightarrow P(B)$
4. Probabilitat d'A després de veure B: posterior $\rightarrow P(A|B)$

Bayes serveix per invertir una probabilitat si algun terme es coneix.

Per exemple:

*$P(A|B)$ és **fàcil** de comptar*

*$P(B|A)$ és **difícil***

Vídeo molt explicatiu del teorema de Bayes: <https://www.youtube.com/watch?v=n11NDXIlcLI&t=153s>

Si apliquem la fórmula de distribució conjunta a Bayes tenim:

$$P(A|B) = \frac{P(B,A)P(A)}{P(B)}$$

$$P(A|B) = \frac{P(A,B)}{P(B)} \text{ substituïm a la fórmula de dalt}$$

$$P(A|B) = \frac{\frac{P(B,A)}{P(A)} \cdot P(A)}{P(B)} = \frac{P(B,A)}{P(B)}$$

$$P(A|B) = \frac{P(B,A)}{P(B)}$$

Exemple Per entendre-ho millor imaginem que tenim 100 emails a on el contingut dels quals només utilitzen 2 paraules. Amb això tindríem:

(oferta, gratis)	SPAM	NO SPAM	Total	Oferta	Gratis	SPAM	NO SPAM	Total
(0,0)	5	40	45	0	0	5	40	45
(0,1)	10	5	15	0	1	10	5	15
(1,0)	20	10	30	1	0	20	10	30
(1,1)	8	2	10	1	1	8	2	10
Total	43	57	100	Total		43	57	100

Marginal de la classe (SPAM/NO SPAM)

- $P(\text{SPAM}) = 43/100 = 0.43$
- $P(\text{NO SPAM}) = 57/100 = 0.57$

Marginal de cada combinació de paraules

- $P(0,0) = P(\text{oferta}=0, \text{gratis}=0) = 45/100 = 0.45$
- $P(0,1) = P(\text{oferta}=0, \text{gratis}=1) = 15/100 = 0.15$
- $P(1,0) = P(\text{oferta}=1, \text{gratis}=0) = 30/100 = 0.30$
- $P(1,1) = P(\text{oferta}=1, \text{gratis}=1) = 10/100 = 0.10$

Com hem dit abans el Teorema de Bayes només s'utilitza per invertir el condicionament. Per tant, si volem saber la $P(\text{SPAM} | x_1, x_2)$ i $P(\text{NO SPAM} | x_1, x_2)$, però és fàcil saber $P(x_1, x_2 | \text{SPAM})$ tenim:

$$P(\text{SPAM} | x_1, x_2) = \frac{P(x_1, x_2, \text{SPAM})}{P(x_1, x_2)}$$

Exemple: Imaginem que volem saber quina probabilitat té un correu de ser SPAM sabent que té **oferta=1** i **gratis=0**?

$$P(\text{SPAM} | 1,0) = \frac{P(1,0, \text{SPAM})}{P(1,0)} = \frac{20/100}{30/100} = \frac{0.2}{0.3} = \mathbf{0.6667}$$

$$P(\text{NO SPAM} | 1,0) = \frac{P(1,0, \text{NO SPAM})}{P(1,0)} = \frac{10/100}{30/100} = \frac{0.1}{0.3} = \mathbf{0.3333}$$

Naive Bayes

Com que en datasets amb moltes característiques és inviable calcular la conjunta real perquè no tenim prou dades o massa combinacions. És a dir hem de poder calcula la probabilitat conjunta de totes les combinacions.

Per exemple, si tenim emails de només 10 paraules la nostra taula de contingència seria de $2^{10} = 1024$ files. I si tenim emails de 100 paraules $2^{100} = 1.267.650.600.228.229.401.496.703.205.376$ files. Per tant, **aquesta taula de contingències és inviable!**

Ara bé, el que si que podem estimar fàcilment són les probabilitats una a una assumint que les característiques són independents entre elles donada la classe.

$$P(X|y) = P(x_1|y) \cdot P(x_2|y) \cdot P(x_n|y) = P(x_1, x_2, \dots, x_n|y) \approx \prod_{i=1}^n P(x_i|y)$$

Ara en comptes de necessitar una taula de 2^n , Només necessitem: $P(x_1|y), P(x_2|y), \dots, P(x_n|y)$ i això si que ho podem calcular fàcilment.

També assumim la independència per la conjunta marginal

$$P(X) = P(x_1, x_2, \dots, x_n) \approx \sum_y P(y) \prod_{i=1}^n P(x_i|y)$$

Si partim del Teorema de Bayes

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$
$$P(y|X) = \frac{\prod_{i=1}^n P(x_i|y) \cdot P(y)}{\sum_y P(y) \prod_{i=1}^n P(x_i|y)}$$

Aquesta ingènua (*naive*), perquè en el món real les variables solen estar correlacionades entre sí, però matemàticament **simplifica** moltíssim i a la pràctica, **funciona sorprenentment bé**, sobretot per text.

Sota aquesta assumpció no cal calcular la distribució condicional conjunta $P(X|y)$ sinó que calcularem la distribució condicional de cada atribut $P(x_i|y)$ respecte la classe (funcions de versemblança independents).

Seguint amb l'exemple anterior :

Marginal de la classe (SPAM/NO SPAM). Probabilitats prèvies $P(y)$

- $P(\text{SPAM}) = 43/100 = 0.43$
- $P(\text{NO SPAM}) = 57/100 = 0.57$

Condicionals simples de cada característica per classe $P(x_i|y)$

SPAM (43)	NO SPAM (57)
$P(x_1=1 \text{SPAM}) = P(\text{oferta}=1 \text{SPAM}) = (20+8)/43=0.651$	$P(x_1=1 \text{NOSPAM}) = P(\text{oferta}=1 \text{NOSPAM}) = (10+2)/57=0.210$
$P(x_1=0 \text{SPAM}) = P(\text{oferta}=0 \text{SPAM}) = (5+10)/43=0.348$	$P(x_1=0 \text{NOSPAM}) = P(\text{oferta}=0 \text{NOSPAM}) = (40+5)/57=0.789$
$P(x_2=1 \text{SPAM}) = P(\text{gratis}=1 \text{SPAM}) = (10+8)/43=0.418$	$P(x_2=1 \text{NOSPAM}) = P(\text{gratis}=1 \text{NOSPAM}) = (5+2)/57=0.1228$
$P(x_2=0 \text{SPAM}) = P(\text{gratis}=0 \text{SPAM}) = (5+20)/43=0.581$	$P(x_2=0 \text{NOSPAM}) = P(\text{gratis}=0 \text{NOSPAM}) = (10+40)/57=0.8771$

Numerador

$$P(1,0|\text{SPAM}) = P(\text{SPAM}) \cdot P(\text{oferta} = 1|\text{SPAM}) \cdot P(\text{gratis} = 0|\text{SPAM})$$

$$P(1,0|\text{SPAM}) = 0.43 \cdot 0.651 \cdot 0.581 = \mathbf{0.1626}$$

$$P(1,0|\text{NO SPAM}) = P(\text{NOSPAM}) \cdot P(\text{oferta} = 1|\text{NOSPAM}) \cdot P(\text{gratis} = 0|\text{NOSPAM})$$

$$P(1,0|\text{NOSPAM}) = 0.57 \cdot 0.210 \cdot 0.8771 = \mathbf{0.1049}$$

Amb aquest resultat ja podríem classificar i indicar que la classe guanyadora és SPAM. Per tant una observació d'oferta=1 i gratis=0 la classificaríem com SPAM.

Denominador

Ara. Si volem treure la probabilitat hem de calcular el denominador del Teorema de Bayes

$$P(X) = P(1,0) = P(1,0|\text{SPAM}) + P(1,0|\text{NOSPAM}) = 0.1626 + 0.1049 = \mathbf{0.2675}$$

$$P(1,0|\text{SPAM}) = P(\text{SPAM}) \cdot P(\text{oferta} = 1|\text{SPAM}) \cdot P(\text{gratis} = 0|\text{SPAM})$$

Teorema de Naive Bayes

$$P(SPAM|1,0) = \frac{0.1626}{0.2675} = 0.607$$
$$P(NO SPAM|1,0) = \frac{0.1049}{0.2675} = 0.392$$

Si ens hi fixem el valor de la probabilitat $P(SPAM|0,1)$ amb Naive Bayes és de 0.607, molt propera a la real que és 0.6667

Laplace Smoothing

Un dels problemes que presenta la fórmula ingènua de Bayes és que si hi algun termes X que no s'ha vist mai el model mai classificarà aquest terme.

Imaginem que tenim una característica (paraula) “premi” que no s'ha vist mai o classificat mai en un correu NO SPAM.

$$P(\text{premi} = 1 | NO SPAM) = P(x_3 = 1 | NO SPAM) = 0/43 = 0$$

Qualsevol **numerador** valdrà 0 i el nostre classificador ja no funcionarà.

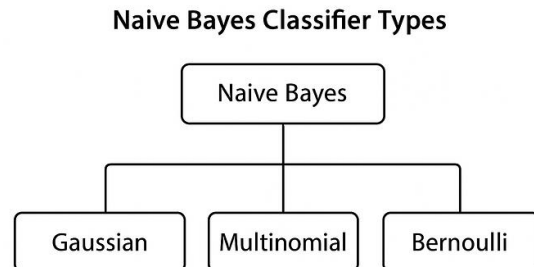
La idea és: “Encara que no hagi vist mai aquesta paraula en SPAM al teu dataset, no vol dir que la probabilitat real sigui exactament 0. Potser simplement encara no ha aparegut.”

Laplace Smoothing (també anomenat **add-one smoothing**) diu: “Fem veure que hem vist **com a mínim una vegada** cada possible valor, encara que en el dataset no hi sigui.”

Tipus de classificadors Naive Bayes

Naive Bayes té diferents variants adaptades als diversos tipus de dades:

- **Gaussian** Naive Bayes: Assumeix que les característiques segueixen una distribució normal; ideal per a dades contínues, com mesuraments o lectures de sensors.
- **Multinomial** Naive Bayes: Treballa amb recomptes discrets, com freqüències de paraules; perfecte per a classificació de documents.
- **Bernoulli** Naive Bayes: Funciona amb característiques binàries, com la presència o absència d'una paraula en un text.



Model	Tipus de dades	Casos típics d'ús
GaussianNB	dades contínues (float)	sensors, mètriques, dades numèriques
MultinomialNB	dades de recompte (counts)	text (bag-of-words, TF-IDF)
BernoulliNB	dades binàries (0/1)	presència/absència, features booleans
CategoricalNB	dades categòriques discretes	categories finites (nivell educatiu, colors, etc.)

Punts forts i febles

Punts forts

- **Ràpid i escalable:** Processa conjunts de dades grans amb molta rapidesa.
- **Necessita poca informació:** Funciona bé fins i tot amb pocs exemples, a diferència de models profunds que en necessiten molts.
- **Amic de l'alta dimensionalitat:** Excel·leix en tasques de text amb milers de característiques.
- **Preparat per multiclasse:** Gestiona de manera natural múltiples categories, com temàtiques de notícies (esports, tecnologia, política).

Exemple: En la classificació de notícies, Naive Bayes assigna articles a temes amb molt poc càlcul, fins i tot quan hi ha milers de paraules.

El punt feble: limitacions

- **Assumpció d'independència:** En el món real, moltes característiques estan correlacionades (per exemple, “machine” i “learning” en text), cosa que pot afectar les prediccions.
- **Problema de probabilitat zero:** Si una característica no apareix en cap exemple d'una classe durant l'entrenament, la seva probabilitat és zero i trenca la predicció.
- **Característiques correlacionades:** Té dificultats quan les característiques depenen fortament entre elles.

Exemple: En filtres de correu brossa, laplace smoothing assegura que una paraula rara com “bitcoin” no malmeti el model.

Decision Trees (Classifier)

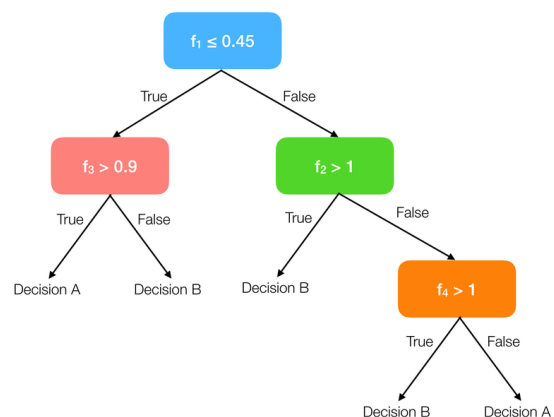
Com s'ha dit a l'apartat de Decision Trees (Regressió) dins dels Models Supervisats de Regressió els arbres de decisió són models que prenen decisions seguin una estructura jeràrquica en forma d'arbre a on cada node representa una pregunta/prova sobre un atribut i cada branca correspon a la resposta de la pregunta. Finalment el camí que segueix una nova observació de l'arrel fina a la fulla determina la seva predicció.

Els arbres de classificació són un subconjunt dels arbres de decisió aplicant quan la variable a predir és categòrica (classe).

Potser són més senzills d'entendre perquè al actuar com a classificador la seva funció és aprendre una sèrie de preguntes seqüencials que permeten decidir en quina classe caurà cada observació.

Per exemple podem pensar en un triatge mèdic:

- Té febre?
 - Sí, té tos seca? Possible pneumònia
 - No, té dolor al pit? Constipat



Per construir un arbre de classificació s'utilitza el mateix mètode de *recursive binary splitting* que s'ha explicat en els arbres de regressió a on es realitzen divisions de l'espai de dades, però existeixen diferents alternatives de criteris de selecció.

Com aprèn un arbre? Idea d'impuresa i homogeneïtat

Com hem dit per construir un arbre cal decidir quina és la millor pregunta que puc fer en cada node per separar millor les classes.

Per decidir-ho, utilitzarem el concepte **d'impuresa del node**, és a dir, com de barrejades estan les classes en aquell node.

- Si un node té totes les observacions d'una mateixa classe direm que és un **node pur** i la seva **impuresa = 0**.
- En canvi si el node té classes al 50-50 direm que és aquell node té **impuresa màxima**.

Aquesta impuresa es pot mesurar amb diverses funcions:

- Classification Error Rate (CER)
- Gini Index
- Cross-entropy
- Chi-quadrat(X^2)

Mesura	Sensibilitat	Ús habitual
Classification Error Rate (CER)	Baixa	Poc usada per fer splits
Gini Index	Alta	CART, sklearn
Cross-entropy	Alta	ID3, C4.5, C5.0
Chi-quadrat (X^2)	Mitjana	CHAID

Classification Error Rate (CER)

És la mesura més simple i es defineix com:

$$E = 1 - \max_k(p_k)$$

On:

- (p_k) : proporció d'exemples de la classe k en el node
- $\max_k(p_k)$: proporció de la classe majoritària

És la proporció d'observacions del node que **no** pertanyen a la classe majoritària

Per exemple si tenim un node a on tenim 80 positius i 20 negatius:

$$E = 1 - 0.8 = 0.2$$

$$E = 1 - 0.2 = 0.8$$

Normalment no s'utilitza perquè no diferencia bé nodes mig purs de nodes molt purs.

Índex de Gini (criteri per defecte a CART i scikit-learn)

$$G = \sum_{k=1}^K p_k(1 - p_k)$$

També es pot expressar com:

$$G = 1 - \sum_{k=1}^K p_k^2$$

L'índex de Gini té un rang de 0 - 0.5:

- Si el **node** és **pur**. **Totes** les observacions del node són de la **mateixa classe**. **Gini = 0**
- Quan més **gran** és la **impuresa** del conjunt, més tendeix l'índex de Gini a 0.5 en un conjunt binari.

El guany d'informació és, per definició, la disminució de l'entropia causada per l'atribut seleccionat, però també es pot relacionar amb una disminució de l'Índex de Gini.

- El **Gini** és una mesura **d'incertesa**, impuresa
 - Gini = 0 → node **pur** (les classes estan ben separades → millor)
 - Gini alt → node **impur** (les classes estan barrejades → pitjor).
- El Gini Gain (guany de Gini), és **quanta impuresa ham eliminat** gràcies al fer l'split.

$$\text{Gini Gain} = G_{\text{pare}} - G_{\text{següent}}$$

- **Gini Gain = gran** → Hem reduït **molt** la impuresa
- **Gini Gain = baix** → Hem reduït **poc** la impuresa

EXEMPLE 1 amb dades numèriques: Volem construir un arbre de decisió utilitzant un data set de 8 **observacions** per classificar dues **classes A i B** i una **característica numèrica**.

CART fa l'split (divisió) per les mitjanes entre valors consecutius. En aquest cas tindrem:

- $2 \text{ i } 3 \rightarrow 2.5, 3 \text{ i } 5 \rightarrow 4, 5 \text{ i } 6 \rightarrow 5.5, 6 \text{ i } 7 \rightarrow 6.5$

Tenim el següent recompte:

- A: 5 (2,3,3,3,6) = 5/8 = 0.625
- B: 3 (5,7,7) = 3/8 = 0.375

$$G_{pare} = 1 - \sum_{k=1}^K p_k^2$$

$$G_{pare} = 1 - (0.625^2 + 0.375^2) = 1 - 0.5312 = \mathbf{0.4688}$$

X1	Classe
2	A
3	A
3	A
3	A
5	B
6	A
7	B
7	B
A	5/8
B	3/8

Per cada split calculem la impuresa resultant i la comparem amb la pare

SPLIT 1: $X_1 \leq 2.5$

Esquerra	Dreta
Valors: [2], classes[A]	Valors: [3,3,3,5,6,7,7], classes[A,A,A,B,A,B,B]
A:1, B:0	A:4, B:3
Gini = 0	Gini = $1 - ((4/7)^2 + (3/7)^2)$
	Gini = $1 - 0.5102$
	Gini = 0.4898

$$\text{Gini resultat } G = \frac{1}{8} \cdot 0 + \frac{7}{8} \cdot 0.4898 = 0.4286$$

$$\text{Gini Gain: } 0.4688 - 0.4286 = 0.0402$$

$$\text{Gini Gain} = \mathbf{0.0402}$$

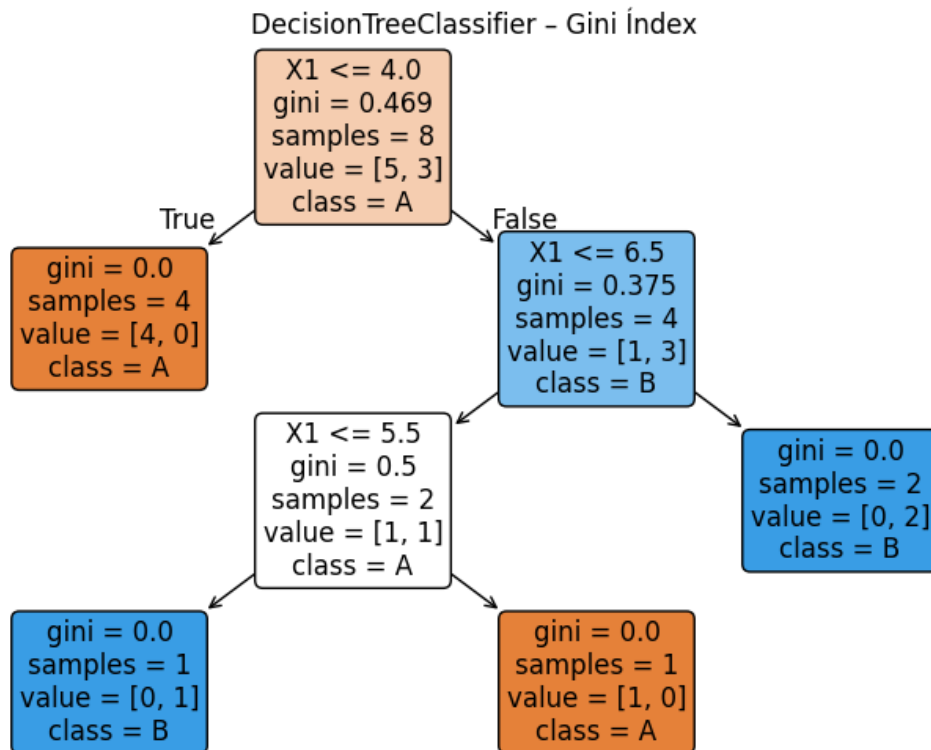
Aquest càlcul el fem per tots els splits restants 4, 5.5 i 7.5 i tenim la taula següent

Split	Gini Pare	Gini	Gini Gain
2.5	0.4688	0.4286	0.0402
4	0.4688	0.1875	0.2813

5.5	0.4688	0.3665	0.1023
6.5	0.4688	0.2085	0.2603

El millor split és el de la característica X és 4.

Per tant, separaríem les dades per aquest bloc



EXEMPLE 2 amb dades categòriques: Volem construir un arbre de decisió utilitzant un dataset de 8 **observacions** per classificar dues **classes A i B** i una **característica categòrica (vermell,blau,verd,negre)**.

X1	Classe
Vermell	A
Blau	A
Blau	B
Verd	A
Negre	B
Vermell	B
Verd	B
Blau	A
A	4/8
B	4/8

En aquest cas fem els splits escollint un subconjunt de categories i calculem el Gini. Compte de si tenim variables categòriques a on tenim moltes categories perquè tindrem molts subconjunts possibles. Per exemple, si tenim K categories hi hauran un total de $2^{K-1} - 1$ *splits possibles*.

- 2 categories → 1 split, 3 categories → 3 splits, 4 categories → 7 splits,...

Tenim el següent recompte:

- A: 4 (Vermell, Blau, Verd, Blau) = 4/8 = 0.5
- B: 4 (Blau, Negre, Vermell, Verd) = 4/8 = 0.5

$$G_{pare} = 1 - \sum_{k=1}^K p_k^2$$

$$G_{pare} = 1 - (0.5^2 + 0.5^2) = 1 - 0.5 = \mathbf{0.5}$$

Per cada split calculem la impuresa resultant i la comparem amb la pare

Tenim 4 categories, per tant, $4^{4-1} - 1 = 7$ *splits*

1. {Vermell} vs {Blau, Verd, Negre}
2. {Blau} vs {Vermell, Verd, Negre}
3. {Verd} vs {Vermell, Blau, Negre}
4. {Negre} vs {Vermell, Blau, Verd}
5. {Vermell, Blau} vs {Verd, Negre}
6. {Vermell, Verd} vs {Blau, Negre}
7. {Vermell, Negre} vs {Blau, Verd}

SPLIT 1: Color $\in \{\text{Vermell}\}$

Esquerra {vermell}	Dreta {blau,verd,negre}
Vermell A:1	Blau A: 2, Blau B: 1
Vermell B: 1	Verd A: 1, Verd B: 1
A:1, B:1	Negre A:0, Negre B:1
Gini = $1 - ((1/2)^2 + (1/2)^2)$	A:3, B:3
Gini = $1 - 0.5$	Gini = $1 - ((3/6)^2 + (3/6)^2)$
Gini = 0.5	Gini = $1 - 0.5$
	Gini = 0.5

$$\text{Gini resultat } G = \frac{2}{8} \cdot 0.5 + \frac{6}{8} \cdot 0.5 = 0.5$$

$$\text{Gini Gain: } 0.5 - 0.5 = 0.0$$

$$\text{Gini Gain} = 0.0$$

SPLIT 5: Color $\in \{\text{Vermell}, \text{Blau}\}$

Esquerra {vermell,blau}	Dreta {verd,negre}
Vermell A:1, Vermell B: 1	Verd A: 1, Verd B: 1
Blau A:2, Blau B: 1	Negre A: 0, Negre B: 1
A:3, B:2	A:1, B:2
Gini = $1 - ((3/5)^2 + (2/5)^2)$	Gini = $1 - ((1/3)^2 + (2/3)^2)$
Gini = $1 - (0.36 + 0.16)$	Gini = $1 - 0.555$
Gini = 0.48	Gini = 0.445

$$\text{Gini resultat } G = \frac{5}{8} \cdot 0.48 + \frac{3}{8} \cdot 0.445$$

$$\text{Gini resultat } G = 0.30 + 0.1668 = 0.4668$$

$$\text{Gini Gain: } 0.5 - 0.4668 = 0.0332$$

$$\text{Gini Gain} = 0.0332$$

I així successivament fins arribar en les 7 splits i mirar quin és el millor Gain.

RESUM – Índex de Gini

Entropia de Shannon (Cross-Entropy / Information Gain)

El concepte d'entropia va tenir origen en la termodinàmica com a mesura de desordre molecular, l'entropia s'aproxima a zero quan les molècules estan quietes i ben ordenades..

Un altre índex per mesurar la impuresa és el Cross-Entropy que es defineix com:

$$H = - \sum_{k=1}^K p_k \log_2(p_k)$$

Característiques:

- És **0** quan el node és **pur**
- És màxima quan totes les classes tenen la mateixa probabilitat, per tant depèn de la K. $1/K$
 - $H = - \sum_{k=1}^K p_k \log_2(p_k) \rightarrow$ si substituïm p_k per $1/K$
 - $H_{max} = \log_2(K)$
- Està profundament connectada amb la **teoria de la informació**

X1	Classe
2	A
3	A
3	A
3	A
5	B
6	A
7	B
7	B
A	5/8
B	3/8

Càlcul del guany

$$Gain = H_{pare} - \left(\frac{N_{esq}}{N} H_{esq} + \frac{N_{dret}}{N} H_{dret} \right)$$

On:

- $H_{esq}, H_{dret} =$ entropies dels fills
- $N_{esq}, N_{dret} =$ mida dels fills
- $N =$ mida total del node

EXEMPLE 1 amb dades numèriques: Utilitzarem el mateix dataset que amb el cas de Gini. 8 observacions per classificar dues classes A i B i una característica numèrica.

Cross Entropy també fa l'split (divisió) per les mitjanes entre valors consecutius. En aquest cas tindrem:

- $2 \text{ i } 3 \rightarrow 2.5, 3 \text{ i } 5 \rightarrow 4, 5 \text{ i } 6 \rightarrow 5.5, 6 \text{ i } 7 \rightarrow 6.5$

Tenim el següent recompte:

- A: 5 (2,3,3,3,6) = $5/8 = 0.625$
- B: 3 (5,7,7) = $3/8 = 0.375$

$$H_{pare} = - \sum_{k=1}^K p_k \log_2(p_k)$$

$$H_{pare} = -0.625 \cdot \log_2(0.625) - 0.375 \cdot \log_2(0.375) = 0.42375 + 0.530625$$

$$H_{pare} = \mathbf{0.954375}$$

Per cada split calculem la impuresa resultant i la comparem amb la pare

SPLIT 1: $X_1 \leq 2.5$

Esquerra	Dreta
Valors: [2], classes[A]	Valors: [3,3,3,5,6,7,7], classes[A,A,A,B,A,B,B]
A:1, B:0	A:4, B:3
$P_A=1/1=1$	$P_A=4/7=0.571$
$P_B=0/1=0$	$P_B=3/7=0.4285$
$H_{esq} = -1 \cdot \log_2(1) - 0 \cdot \log_2(0)$	$H_{dret} = -0.571 \cdot \log_2(0.571) - 0.428 \cdot \log_2(0.428)$
$H_{esq} = 0$	$H_{dret} = 0.4616 + 0.524$
	$H_{dret} = 0.9856$

Pesos:

- Node esquerre: $N_E = 1$
- Node dret: $N_D = 7$
- Total: $N = 8$

$$H_{després} = \frac{1}{8}H_E + \frac{7}{8}H_D$$

Substituint:

$$H_{després} = 0.125(0) + 0.875(0.9856)$$

$$H_{després} = 0.8624$$

SPLIT 2: $X_2 \leq 4$

Esquerra	Dreta
Valors: [2,3,3,3], classes[A,A,A,A]	Valors: [5,6,7,7], classes[B,A,B,B]
A:4, B:0	A:1, B:3
$P_A = 4/4 = 1$	$P_A = 1/4 = 0.25$
$P_B = 0/4 = 0$	$P_B = 3/4 = 0.75$
$H_{esq} = -1 \cdot \log_2(1) - 0 \cdot \log_2(0)$	$H_{dret} = -0.25 \cdot \log_2(0.25) - 0.75 \cdot \log_2(0.75)$
$H_{esq} = 0$	$H_{dret} = 0.5 + -0.3112$
	$H_{dret} = 0.1887$

Pesos:

- Node esquerra: $N_E = 4$
- Node dret: $N_D = 4$
- Total: $N = 8$

$$H_{després} = \frac{4}{8}H_E + \frac{4}{8}H_D$$

Substituint:

$$H_{després} = 0.5(0) + 0.5(0.1887)$$

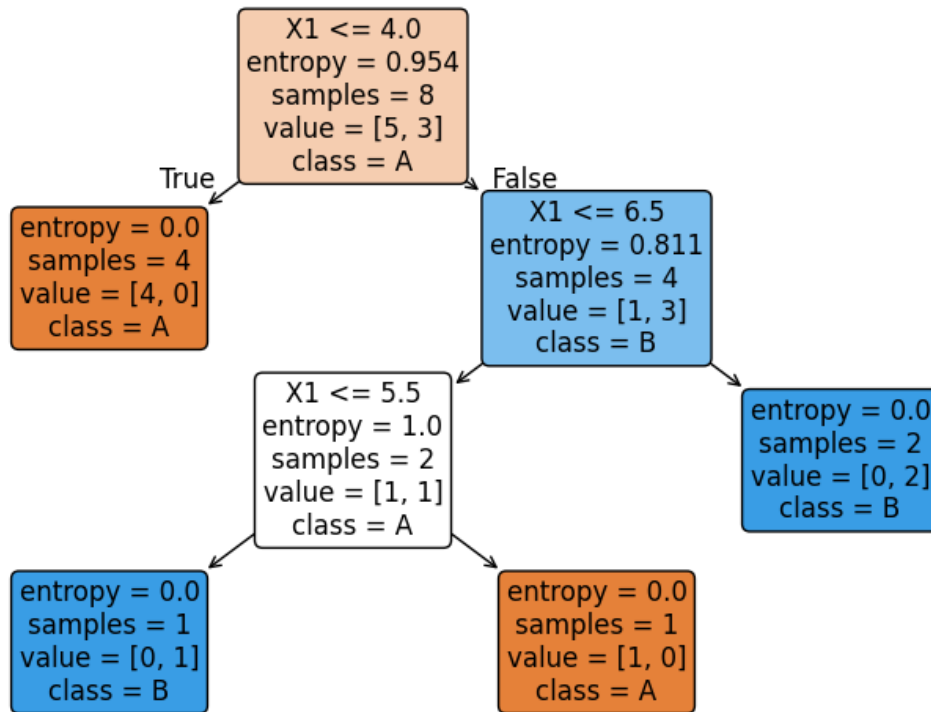
$$H_{després} = 0.09435$$

Aquest càlcul el fem per tots els splits restants 4, 5.5 i 7.5 i tenim la taula següent

Split	H Pare	H	Guany
2.5	0.954375	0.8624	0.09
4	0.954375	0.1875	0.766875
5.5	0.954375	...	...
6.5	0.954375	...	...

De moment el millor split és el de la característica X és 4.

DecisionTreeClassifier - Cross Entropy



Quan utilitzar Gini o Entropy?

En termes generals utilitzarem Gini ja que és un mètode molt més ràpid. Normalment és el valor utilitzat per defecte en els models basats en CART (Random ForestClassifier i ExtraTreesClassifier)

Degut a que **les diferències són mínimes preferim simplicitat i rapides i per això Gini és un molt bon “default”**

Cross Entropy l'utilitzarem si volem una interpretació més probabilística. En certs dataset, l'entropia pot donar splits lleugerament diferents, i ser una mica més sensible quan les classes estan molt barrejades, però les diferències en accuracy són mínimes.

Cross-Entropy ha de calcular logaritmes que el fa molt més lent.

La impuresa de Gini tendeix a aïllar la classe més freqüent en la seva pròpia branca de l'arbre, en canvi la entropia tendeix a produir arbres més equilibrats. [Veure article](#)

Random Forest Classifier

Tal i com s'ha comentat en l'apartat de Random Forest for Regression, els Random Forest for Classifier funcionen igual i el seu objectiu és el mateix.

L'objectiu és reduir els inconvenients d'un sol arbre (com el sobreajustament o alta variància) i aconseguir un model més estable i precís.

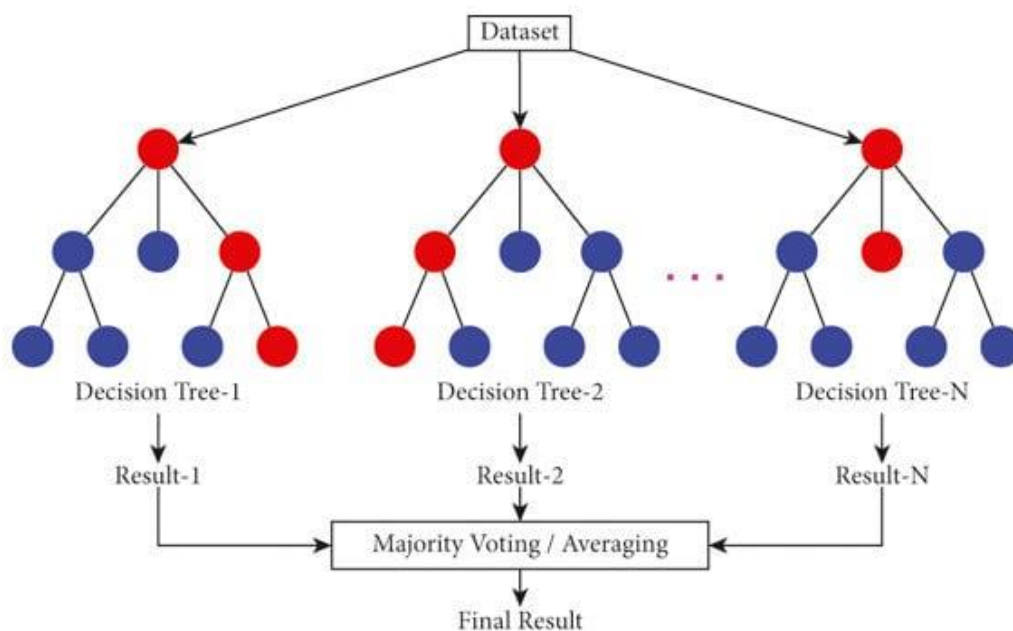
Cada arbre **s'entrena independentment** amb una **mostra aleatòria** de les dades originals, obtinguda mitjançant una tècnica anomenada **bootstrapping**.

- Cada arbre veu un **subconjunt lleugerament diferent** de les dades d'entrenament.
- **Algunes observacions poden repetir-se**, i d'altres potser no apareixen en absolut en aquell arbre concret.

Un sol arbre té alta variància, si canvies una mica les dades d'entrenament, l'arbre pot canviar molt i tendeix a sobre ajustar-se si no es controla bé la profunditat i els `min_samples_leaf`.

En el cas de regressió cada arbre genera una predicció numèrica i el model final calcula la **mitjana** d'aquestes prediccions

Els arbres de decisió donen el resultat d'una classe i es retorna la classe més votada. Si hi ha empat s'utilitza la classe de l'índex menor.



KNN (K-Nearest Neighbors)



El seu nom (Nearest Neighbors) ja delata el funcionament del model. Aquest model opera amb el principi de proximitat dels exemplars de tal manera que fa prediccions mirant els exemples més semblants/propers que ha vist abans.

És un algorisme supervisat, no paramètric que s'utilitza bàsicament per **classificació** tot i que es pot utilitzar també per **regressió**.

Si volem establir una analogia amb la vida quotidiana podríem dir que és un algorisme gregari amb el sentit que pren decisions en funció dels seus veïns. Per exemple si tots els meus veïns van vestits de màniga curta jo també seré una persona que porta màniga curta.

Un classificador *KNN* és un model d'aprenentatge automàtic que fa prediccions basant-se en la classe majoritària dels *K* punts de dades més propers en l'espai de característiques. L'algorisme KNN assumeix que les coses semblants existeixen a prop les unes de les altres, fet que el fa intuïtiu i fàcil d'entendre.

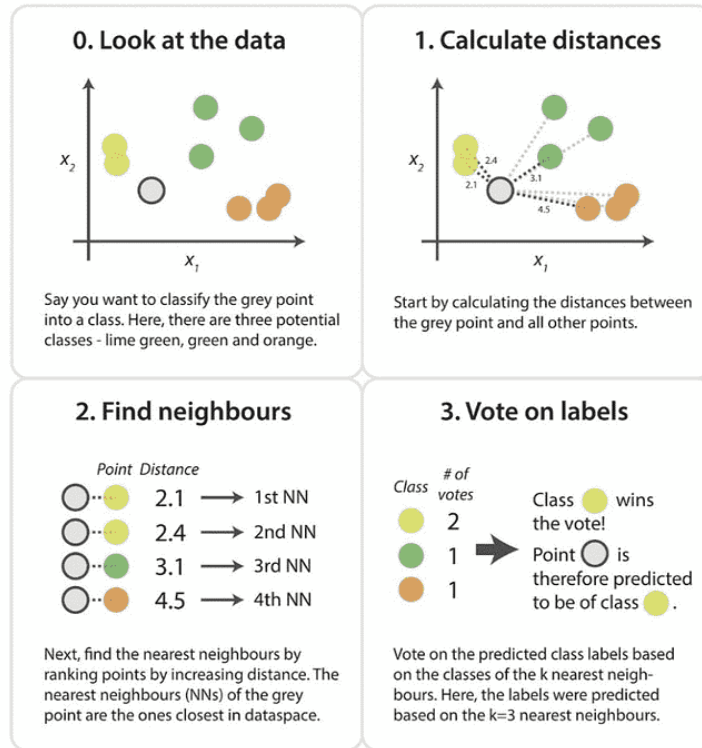
Si estem parlant de **classificació**, assignem la classe majoritària dels veïns més propers. En problemes de **regressió** el valor resultat és la mitjana aritmètica dels veïns més propers.

Es diu que és un model "*lazy learning*" o "*instance-based learning*" perquè és un model que no hi ha fase d'entrenament com en els altres models sinó que "memoritza" les dades d'entrenament.

Les idees inicials se li atribueixen a Evelyn Fix i Joseph Hodges en el seu [article](#) publicat el 1951. Thomas Cover amplia el seu concepte a la seva investigació amb el nom de "[Nearest Neighbor Pattern Classification](#)".

Funcionament

1. Escollim o **establim el paràmetre K**. Aquest valor es decideix **empíricament** i no teòricament.
2. **Calcula la distància** entre el nou punt de dades i **tots** els punts del conjunt d'entrenament.
3. **Selecciona els K veïns més propers** segons aquestes distàncies.
4. **Predicció**
 - Classificació: Assignem la **classe majoritària** del *K* veïns
 - Regressió: Fem la **mitjana** o mitjana ponderada.



Tipus de distàncies

De distàncies n'hi ha de diferent tipus, però les més habituals són:

Distància Euclidiana

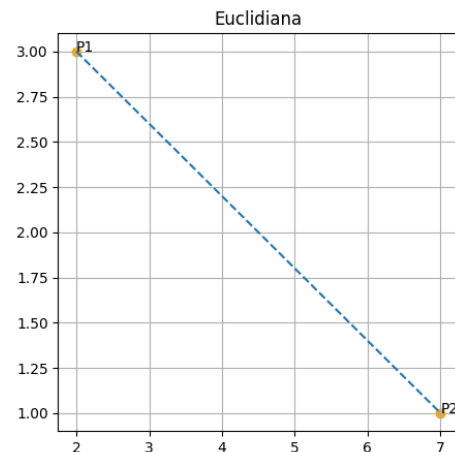
És la distància més utilitzada en models KNN. És la distància de la línia recta entre dos punts .

$$\text{distància } d_{2D} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$\text{distància } d_{3D} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

En **N** dimensions:

$$\text{distància } d_{ND} = \sqrt{\sum_{i=1}^N (p_{2,i} - p_{1,i})^2}$$



Distància Manhattan

Aquesta distància es mesura com el valor absolut entre dos punts i si només poguéssim moure'ns horitzontalment i verticalment. Rep el nom de Manhattan precisament perquè els seu mapa de carrers és perpendicular.

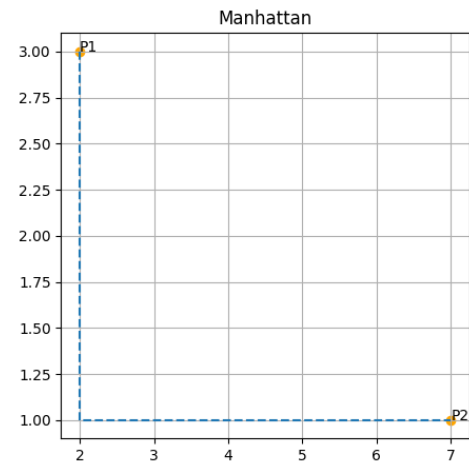
$$\text{distància } d_{2D} = |x_2 - x_1| + |y_2 - y_1|$$

$$\text{distància } d_{3D} = |x_2 - x_1| + |y_2 - y_1| + |z_2 - z_1|$$

En **N** dimensions:

$$\text{distància } d_{ND} = \sum_{i=1}^N |p_{2,i} - p_{1,i}|$$

És molt útil en dades amb valors molt dispersos (*sparse*).



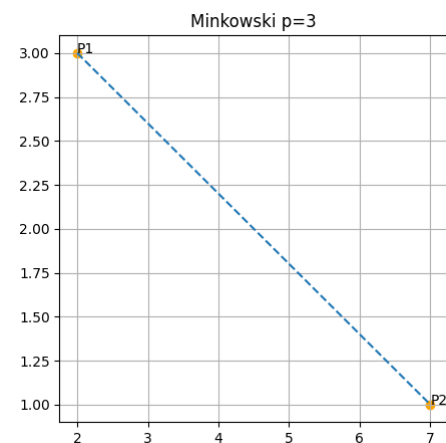
Distància Minkowski

Aquesta distància és la forma generalitzada de les altres mètriques de distància Euclidiana i Manhattan.

El paràmetre t permet la creació d'altres mètriques de distància.

Euclidiana $\rightarrow t=2$ i Manhattan $\rightarrow t=1$

$$\text{distància } d_{ND} = \left(\sum_{i=1}^N |p_{2,i} - p_{1,i}|^t \right)^{\frac{1}{t}}$$



A mesura que **augmentem la t domem més pes a les dimensions amb grans**. Això pot exagerar la separació en direccions dominants.

Anem a veure un **exemple** per entendre-ho millor: **Punt A (0,0), Punt B (1,5)**. Calcularem Minkowski per varis valors de t

Valor de t	Càlcul		resultat
$t = 1$ (manhattan)	$d_1 = 1 - 0 + 5 - 0 $	$d_1 = 1 - 0 + 5 - 0 $	6
$t = 2$ (euclidiana)	$d_2 = (1 - 0 ^2 + 5 - 0 ^2)^{\frac{1}{2}}$	$d_2 = (1 + 25)^{\frac{1}{2}}$	5,09
$t = 4$	$d_4 = (1 - 0 ^4 + 5 - 0 ^4)^{\frac{1}{4}}$	$d_4 = (1 + 625)^{\frac{1}{4}}$	5,52
$t = 10$	$d_{10} = (1 - 0 ^{10} + 5 - 0 ^{10})^{\frac{1}{10}}$	$d_{10} = (1 + 9765625)^{\frac{1}{10}}$	5,999
Veiem que el valor s'apropa a 6 a mesura que anem augmentant el valor de t. La distància queda pràcticament determinada només per la dimensió amb la diferència més alta.			

Distància Chebyshev

La distància de Chebyshev és quan la t del Minkowski tendeix a infinit i és dona distància màxima de desplaçament en qualsevol dimensió.

Distància Hamming

La distància de Hamming és un tipus de distància que només s'utilitza en **dades categòriques**, discretes o binàries.

Data A: 1 1 1 0 0 1 0 1 0 1 0 1 1 0 0 0 0 1 0 0 1 0 0 1 1 1 0 0

Data B: 1 1 1 0 0 1 0 1 0 1 0 1 1 0 0 0 0 1 0 0 1 0 0 1 1 1 0 0

Hamming Distance between Data A and Data B = 1

$$\text{distància } d_H = \sum_{i=1}^N [p_{2,i} \neq p_{1,i}]$$

On: $[p_{2,i} \neq p_{1,i}]$ val: **1** si són **diferents** i **0** si són **iguals**

Exemple 1:

$p_1 = (1, 0, 1, 1)$

$p_2 = (1, 1, 0, 1)$

Comparem posició per posició i tenim:

$$\text{distància } d_H = 0 + 1 + 1 + 0 = 2$$

Exemple 2:

p1 = (ACGTAC)

p2 = (AGGCAT)

Comparem posició per posició i tenim:

$$\text{distància } d_H = 0 + 1 + 0 + 1 + 0 + 1 = 3$$

Exemple 3

p1 = (vermell, SUV, gasolina)

p2 = (blau, SUV, elèctric)

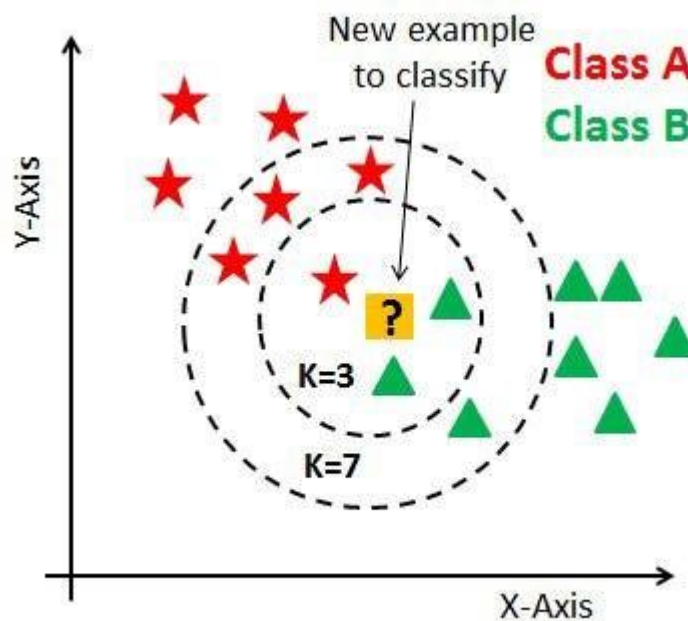
Comparem posició per posició i tenim:

$$\text{distància } d_H = 1 + 0 + 1 = 2$$

Quin valor de K escollim?

Com s'ha dit, el valor de K no és una decisió teòrica sinó empírica i depèn de molts factors:

- La mida, soroll o distribució del teu dataset
- El tipus de problema (classificació/regressió).
- Com d'estable o "local" volem que siguin les prediccions.



K massa petit   (és com preguntar l'opinió **només** al teu veí. Pot ser que tingui un gust molt particular i no representi a la gent del teu barri).

- El model reacciona molt a peits canvis
- És molt sensible al soroll
- Sembla que memoritza les dades i pot caure amb **overfitting**

K massa gran   (és com preguntar l'opinió a tot el país. Segurament la majoria no viu en el teu poble/barri i no el coneixen prou).

- El model és massa general
- Perd detall local
- Pot provar **underfitting**

Un bon **punt de partida** és calcular l'arrel quadrada $K \approx \sqrt{N}$ de la quantitat d'observacions del teu dataset.

- 400 mostres K=20
- 10000 mostres K=100

Per classificació és millor utilitzar valors de K imparells per evitar empats.

Per regressió s'acostuma a necessitar un K més gran per suavitzar el soroll.

En un cas professional cal utilitzar el **cross-validation** mitjançant **GridSearchCV**

```
# PREPROCESSAT
preprocessor = ColumnTransformer(
    transformers=[
        ("cat", OneHotEncoder(handle_unknown="ignore"), X.columns)
    ]
)
pipeline = Pipeline(
    steps=[
        ("preprocess", preprocessor),
        ("knn", KNeighborsClassifier())
    ]
)
```

```

param_grid = {
    "knn__n_neighbors": [1, 3, 5, 7, 9],
    "knn__metric": ["hamming", "manhattan"]
}
grid = GridSearchCV(
    pipeline,
    param_grid=param_grid,
    cv=5,
    scoring="f1" #accuracy, recall, f1
)

grid.fit(X_train, y_train)

print("Millors hiperparàmetres:", grid.best_params_)
print("Millor score:", grid.best_score_)

```

Resum

A diferència de molts altres algorismes d'Aprenentatge Automàtic, **KNN és senzill d'implementar i intuïtiu**. És flexible (ofereix moltes mètriques de distància per triar), **evoluciona** a mesura que es descobreixen **noves dades** i només té **un únic hiperparàmetre** per ajustar (el valor de "k"). Pot detectar **dades** distribuïdes de manera **lineal o no lineal** i, com que és **no paramètric, no requereix complir cap supòsit** per aplicar-lo (a diferència dels models de regressió lineal, que en tenen molts que les dades han de satisfer abans de poder ser utilitzats).

Com qualsevol algorisme, té alguns inconvenients. L'elecció de "k" i de la mètrica de distància és crítica per al seu rendiment i s'ha d'ajustar amb cura. KNN també és molt sensible als valors atípics (*outliers*) i als conjunts de dades desequilibrats. A més, com que tot el que fa l'algorisme és emmagatzemar les dades d'entrenament per fer les seves prediccions, **les necessitats de memòria creixen linealment** amb el nombre de punts de dades que proporcionen per entrenar.

KNN és sens dubte un algorisme que val la pena aprendre. Tant si és per gestionar dades absents com per predir similituds en imatges, és una eina imprescindible!

ANNEX

REFERÈNCIES

- Aurélien Géron (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
- <https://medium.com/@vishalgimhan/types-of-machine-learning-approaches-part-1-3-ml02-0042e443d989>
- Continguts de la docent Cristina Gómez de IES de Teis Vigo
- <https://cienciadedatos.net/documentos/py14-ridge-lasso-elastic-net-python>
- https://cienciadedatos.net/documentos/py09_gradient_boosting_python
- [Aprende IA con Ligdi Gonzalez](#): Obtenció d'alguns gràfics
- https://dcain.etsin.upm.es/~carlos/bookAA/02.1_MetodosdeClasificacion-Naive-Bayes.html
- Naive Bayes: <https://medium.com/@saha.soumyadeep90/zero-to-hero-master-naive-bayes-with-code-and-intuition-f3a55efafbd8>
- Decision Tree Classifier: <https://medium.com/@sachinsoni600517/mastering-decision-tree-models-for-smarter-decisions-451ef8b75836>
- Random Forest / Decision Tree: <https://mlarchive.com/machine-learning/decision-trees-and-random-forest-all-you-need-to-know/>
- KNN-NearestNeighbours: <https://medium.com/@pacosun/dont-underestimate-k-nearest-neighbours-ae4b7b549ef4>
-